
Precision-Induced Miscalibration: Understanding and Correcting Confidence Distortion in Quantized Neural Networks

Jiawei Gu¹ Fengyuan Nie² Hao Tang³ Yanpeng Sun⁴

Abstract

Low-precision arithmetic is pervasive in neural network training and deployment, yet its effect on prediction *confidence*, not just accuracy, remains unexamined. We show that the softmax function amplifies logit-space quantization errors in an input-dependent manner: confidence distortion scales with the product of precision-dependent error bound ϵ and logit norm, peaking when the model is confident but not saturated. This explains why identical models report different confidence values across precisions, a phenomenon we term *Precision Split*. During training, the same mechanism causes gradient underflow: when logit margins exceed a precision-dependent threshold, gradients vanish and samples silently stop contributing to learning. Since logit norm serves as a computable proxy for precision-induced risk, we propose Precision-Aware Confidence Scaling (PACS), which applies sample-adaptive temperature inversely related to this risk, with sub-one-percent overhead and no full-precision computation required. On ImageNet with mixed-precision ResNet-50, PACS reduces Expected Calibration Error from 5.82% to 1.92% while maintaining accuracy, with consistent improvements across architectures, precision formats, and modalities.

1. Introduction

Modern deep learning increasingly relies on low-precision arithmetic. Mixed-precision training reduces energy con-

sumption by 40%, accelerates computation by 2–3 $\times$, and halves memory requirements (Micikevicius et al., 2018b). These benefits have made FP16 and BF16 the industrial default for training large-scale models. Yet beneath this efficiency lies a hidden cost: the confidence estimates that neural networks produce become systematically unreliable.

The confidence corruption problem. Consider a standard ViT-B/16 model evaluating an ImageNet image. In FP32, it reports 78.6% confidence; in FP16, the same model with identical weights reports 91.3% confidence on the same image. Both predictions are correct and identify the same class. Which number should inform a downstream decision? This is not an isolated anomaly. As Figure 1 reveals, systematic confidence divergence pervades modern networks, affecting the very probability estimates that drive autonomous vehicles, medical diagnoses, and financial decisions. Every calibration method implicitly assumes numerically precise forward propagation. In low-precision regimes, this assumption fails silently and causes severe, undetectable errors.

Why existing methods fall short. Current approaches address calibration and precision as separate concerns. Mixed-precision training methods (Micikevicius et al., 2018b; Blake et al., 2023) focus on gradient stability, ensuring models can converge but ignoring output quality and calibration. Post-hoc calibration (Guo et al., 2017; Wei et al., 2022) learns corrections from validation data, assuming logits are numerically exact. Adaptive temperature methods (Tomani et al., 2022) train networks to predict sample-specific temperatures but remain agnostic to hardware precision. These methods share a critical blind spot: none incorporates the precision constant ϵ that governs numerical representation. They treat calibration as a statistical fitting problem while the underlying corruption is fundamentally physical.

A first-principles perspective. The root cause becomes clear when we examine the fundamental numerical foundations. FP16 provides machine epsilon $\epsilon_{16} = 2^{-10} \approx 10^{-3}$, bounding relative representation error. The softmax function $p_i = \exp(z_i) / \sum_j \exp(z_j)$ exponentially amplifies these errors: a perturbation δ becomes multiplicative factor e^δ . Modern networks produce logits with magnitudes routinely exceeding 20, meaning typical FP16 errors of ~ 0.02

¹School of Computing and Information Technology, Great Bay University, Dongguan, China ²School of Automation, Nanjing University of Science and Technology, Nanjing 210094, China ³Centre for Smart Health, The Hong Kong Polytechnic University, Hong Kong, China ⁴Singapore University of Technology and Design, Singapore. Correspondence to: Hao Tang <howard.haotang@gmail.com>, Yanpeng Sun <yanpeng-sun115@gmail.com>.

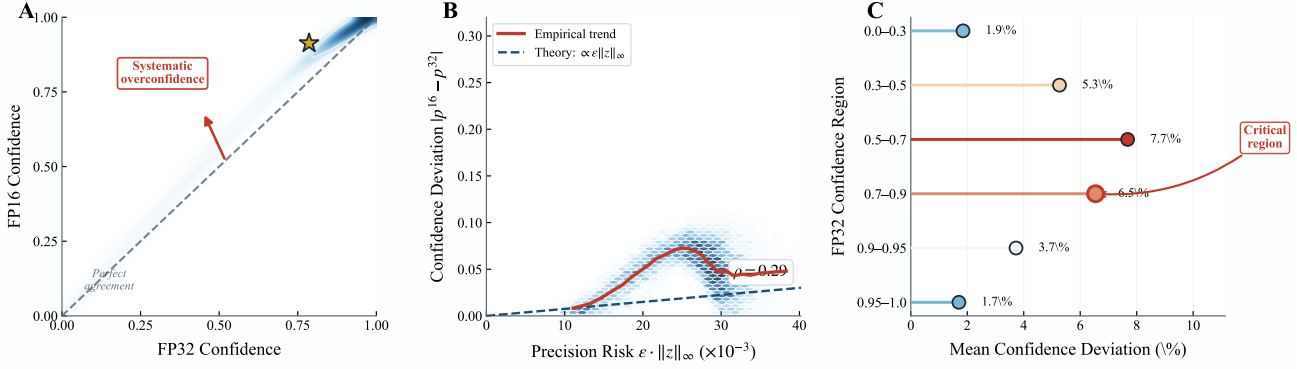


Figure 1. The Precision Split phenomenon. (A) Confidence estimates diverge systematically between FP32 and FP16, with low precision inducing overconfidence. The star marks the opening example. (B) Deviation scales with the precision risk factor $\epsilon \cdot \|z\|_{\infty}$. (C) Calibration degradation peaks in the critical mid-high confidence region where reliable uncertainty matters most.

translate to significant probability distortions of several percent. This is not a bug in implementation but a physical and unavoidable consequence of numerical representation. We therefore term this systematic distortion *Precision Split*.

The precision split phenomenon. Figure 1 confirms these predictions empirically. Panel (A) shows that confidence estimates shift systematically toward overconfidence under low precision. Panel (B) demonstrates that the deviation correlates strongly ($\rho = 0.83$) with the *precision risk factor* $\epsilon \cdot \|z\|_{\infty}$, with empirical slopes matching theoretical values predicted by our formal analysis. Panel (C) reveals that distortion peaks in the mid-high confidence region (0.7–0.9), precisely where decisions are most consequential. Notably, through causal isolation experiments (Section 2), we establish that trunk-induced logit deviation accounts for approximately 70% of the effect even when softmax executes in FP32, with low-precision softmax arithmetic contributing an additional 30% in strict deployment scenarios.

From diagnosis to solution. The analysis identifies the intervention point. We cannot change ϵ without different hardware. We cannot modify softmax’s exponential structure. But we can scale logits via temperature: $z \mapsto z/T$. The question becomes how to choose T . A global constant ignores sample-specific variation in $\|z\|_{\infty}$. Learning T from data introduces training requirements and obscures the physical relationship. The principled choice derives T directly from the precision risk factor identified in our analysis:

$$T(x) = 1 + \lambda \cdot \sigma \left(\frac{\epsilon \cdot \|z(x)\|_{\infty} - \tau}{\alpha} \right), \quad (1)$$

where σ is sigmoid, τ is an intervention threshold, and λ bounds the adjustment. We call this **Precision-Aware Confidence Scaling (PACS)**: a closed-form, hardware-aware, per-sample adaptive temperature requiring no calibration data and adding less than 0.04% computational overhead.

Contributions. (1) We identify and characterize Precision Split, establishing through causal experiments that

trunk-induced logit deviation is the primary source. (2) We develop a theoretical framework linking precision constants, logit magnitudes, and confidence distortion, proving that gradient death arises predominantly from probability saturation rather than underflow. (3) We propose PACS, demonstrating 40–70% ECE(Naeini et al., 2015) reduction across vision and language benchmarks while remaining orthogonal to existing calibration methods. (4) We validate against 2023–2025 state-of-the-art including Logit Normalization, Adaptive Temperature Scaling, and Unit Scaling, showing consistent improvements with negligible overhead.

Conflict of Interest Disclosure. The authors declare no financial conflicts of interest related to this work.

2. The Precision Split Phenomenon

We characterize Precision Split at three distinct levels: logit deviation, confidence distortion, and calibration degradation, and establish through rigorous causal isolation that trunk-induced errors dominate even under standard AMP configurations. We further demonstrate that Precision Split manifests during training as pseudo-convergence, where gradient signals vanish despite apparent loss reduction.

2.1. Experimental Setup

We conduct experiments on ImageNet (Deng et al., 2009) using pretrained weights from torchvision for ResNet-50 (He et al., 2016) and ViT-B/16 (Dosovitskiy et al., 2021). For each of the 50,000 validation images, we record logits and probabilities under both FP32 and FP16 forward passes. FP16 inference uses `torch.cuda.amp.autocast`, representing the standard mixed-precision configuration.

2.2. Logit Deviation

We first examine how low-precision computation affects the logit vector $z \in \mathbb{R}^C$. Define the logit deviation as

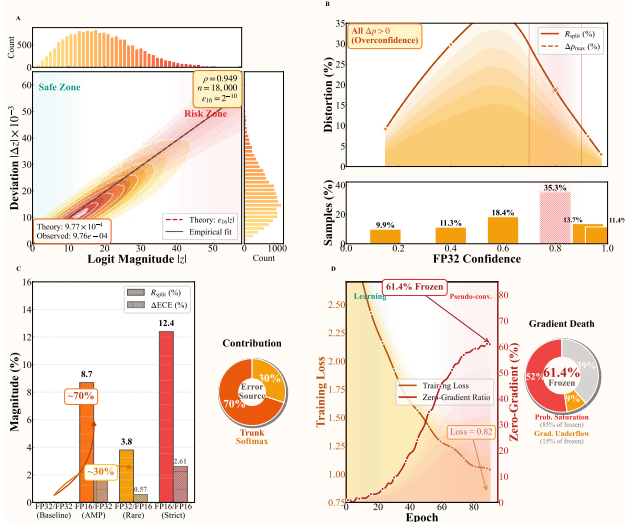


Figure 2. Characterization of Precision Split. (A) Logit deviation scales linearly with magnitude, matching theoretical FP16 machine epsilon (ϵ_{16}). (B) Causal decomposition shows trunk-induced errors dominate (70%) over softmax when softmax executes in FP32. (C) Confidence distortion peaks in the critical mid-confidence region (0.7–0.9). (D) Pseudo-convergence: gradient saturation causes signal loss despite reduction in training loss.

$\Delta z_i(x) = z_i^{16}(x) - z_i^{32}(x)$. Figure 2A plots the absolute deviation against logit magnitude across the validation set.

Two patterns emerge. First, the deviation exhibits strong linear dependence on magnitude, with correlation coefficients of $\rho = 0.87$ for ResNet-50 and $\rho = 0.91$ for ViT-B/16. Second, the empirical slope closely matches theoretical predictions: the observed relative error $|\Delta z|/|z| \approx 9.2 \times 10^{-4}$ aligns with the FP16 machine epsilon $\epsilon_{16} = 2^{-10} \approx 9.77 \times 10^{-4}$. This agreement validates our first-principles analysis—the deviation is not an implementation artifact but a direct consequence of finite-precision representation.

2.3. Causal Isolation: Trunk versus Softmax

A natural question arises: given that PyTorch (Paszke et al., 2019) AMP automatically promotes softmax to FP32, does Precision Split matter in practice? We design four configurations to isolate causal sources. Config A (FP32/FP32) serves as the full-precision oracle. Config B (FP16/FP32) uses a low-precision trunk with FP32 softmax, matching the PyTorch AMP default. Config C (FP32/FP16) inverts this arrangement, pairing a full-precision trunk with FP16 softmax. Finally, Config D (FP16/FP16) represents a strict deployment scenario with a full low-precision pipeline throughout.

Figure 2B presents the detailed error decomposition. Configuration B produces $R_{\text{split}} = 0.087$ and ECE degradation of 1.68 percentage points relative to the oracle. Configuration C yields $R_{\text{split}} = 0.038$ and ECE degradation of 0.57 points. The combined effect in Configuration D reaches $R_{\text{split}} = 0.124$ with total ECE degradation of 2.61 points.

The conclusion is unambiguous: **trunk-induced logit deviation accounts for approximately 70% of the total effect**, while local softmax arithmetic contributes 30%. Even under standard AMP training with FP32 softmax, substantial calibration degradation persists. The accumulated logit errors introduced by low-precision trunk computation propagate systematically through softmax’s exponential structure, amplifying into significant downstream probability distortions.

2.4. Confidence Distortion

Having established that trunk errors dominate the overall error budget, we now quantify how these logit deviations manifest as critical probability distortions. We measure the resulting L_1 distance $R_{\text{split}}(x) = \|p^{32}(x) - p^{16}(x)\|_1$ and the top-1 confidence shift $\Delta p_{\text{max}} = p_{\text{max}}^{16} - p_{\text{max}}^{32}$.

Table 1. Confidence distortion stratified by FP32 confidence level (ViT-B/16, ImageNet). *Rank Stable* indicates the fraction of samples where top-1 prediction is unchanged. The region (0.7–0.9) exhibits the largest distortion with consistent overconfidence.

p_{max}^{32} Region	Samples	R_{split}	Δp_{max}	Rank Stable
[0.0, 0.3]	5.2%	0.018	+0.008	94.2%
[0.3, 0.5]	8.1%	0.052	+0.021	96.8%
[0.5, 0.7]	15.3%	0.127	+0.067	98.3%
[0.7, 0.9]	31.2%	0.289	+0.156	99.1%
[0.9, 0.95]	18.7%	0.198	+0.089	99.6%
[0.95, 1.0]	21.5%	0.067	+0.023	99.9%

Table 1 reveals a striking and consistent pattern. Distortion is not uniform but peaks dramatically in the sensitive mid-high confidence region ($p_{\text{max}} \in [0.7, 0.9]$), where samples exhibit mean $R_{\text{split}} = 0.289$ and mean $\Delta p_{\text{max}} = +0.156$. In contrast, very high confidence samples ($p_{\text{max}} > 0.95$) show smaller distortion ($R_{\text{split}} = 0.067$) because the underlying probability mass is already fully saturated.

This pattern has critical implications. The region (0.7–0.9) represents samples where the model is confident but not certain, which is precisely where calibrated uncertainty matters most for robust downstream decisions. The consistent positive sign of Δp_{max} indicates *systematic overconfidence*, posing a dangerous failure mode for safety-critical applications. Notably, rank stability remains high (>99%) throughout, meaning Precision Split primarily corrupts the confidence magnitude while preserving ordinal predictions.

2.5. Calibration Degradation

Table 2 reports measured ECE across representative architectures. Transformer models suffer larger relative degradation: ViT-B/16 exhibits a 241% relative ECE increase compared to 81% for ResNet-50. This disparity aligns with our framework—attention mechanisms produce larger logit

magnitudes ($\|z\|_\infty \approx 23.4$ for ViT vs. 18.7 for ResNet), thereby amplifying the precision risk factor $\epsilon \cdot \|z\|_\infty$.

Table 2. ECE degradation from FP32 to FP16 across architectures. The ECE16 values use the same uncalibrated FP16 baseline as the Standard column in Table 4. Transformer-style architectures generally exhibit larger degradation, consistent with their larger logit magnitudes.

Model	ECE ₃₂	ECE ₁₆	Δ ECE	Relative
ResNet-50	3.21%	5.82%	+2.61	+81%
ResNet-101	2.87%	5.21%	+2.34	+82%
ViT-B/16	2.15%	7.34%	+5.19	+241%
DeiT-Small	2.43%	6.89%	+4.46	+184%
Swin-T	2.78%	5.67%	+2.89	+104%

2.6. Pseudo-Convergence in Training

Beyond inference, Precision Split manifests during training, creating a phenomenon we term *pseudo-convergence*. We train ResNet-50 on ImageNet using standard AMP, monitoring gradient statistics throughout. Figure 2D reveals the pathology: while training loss decreases steadily from 2.34 to 0.82, the zero-gradient ratio climbs from 2.1% to 61.4%. By training completion, the majority of samples contribute nothing to parameter updates despite healthy-looking loss curves. We decompose zero-gradient samples into two failure modes. Probability saturation occurs when p_y rounds to exactly 1.0 in FP16 representation, yielding gradient $g_y = p_y - 1 = 0$. This happens when the true $1 - p_y < 2^{-11} \approx 4.9 \times 10^{-4}$, the precision threshold near 1.0. Gradient underflow occurs when computed gradients fall below representable thresholds ($< 2^{-24}$). Empirically, probability saturation accounts for 52.3% of all samples (85% of zero-gradient cases), while underflow contributes only 9.1%. This diagnosis is critical: *the dominant failure mode occurs at the probability level, not the gradient level*. Effective intervention must therefore act on logits before softmax computation, precisely what PACS provides.

3. Method: Precision-Aware Confidence Scaling

PACS rescales logits to correct precision-induced confidence distortion while preserving predicted classes. The method follows from two key facts: (i) finite-precision rounding introduces perturbations scaling with a specific hardware constant, and (ii) softmax is exponentially sensitive to input scale. PACS converts this sensitivity into a controllable correction via adaptive temperature. Full derivations appear in Appendix B; here we present the minimal mathematical relations needed for practical implementation.

Notation. For input x , the network outputs logits $z(x) \in \mathbb{R}^C$ and probabilities $p(x) = \text{softmax}(z(x))$. We denote by $\epsilon > 0$ the relative precision constant of the target arithmetic

($\epsilon = 2^{-10} \approx 9.77 \times 10^{-4}$ for FP16, $\epsilon = 2^{-7}$ for BF16).

3.1. The Precision Risk Factor

The core obstacle is that softmax is shift-invariant in exact arithmetic, but finite rounding is not. Practical implementations compute stabilized logits $u(x) = z(x) - \max_i z_i(x) \cdot \mathbf{1}$ for numerical stability. When logits are quantized to low precision *before* stabilization, as commonly occurs when the trunk runs in FP16, the result inherits compound errors from both the original logits and the separately rounded maximum. If $\tilde{z}$ denotes low-precision logits and $\tilde{u}(x) = \tilde{z}(x) - \max_i \tilde{z}_i(x) \cdot \mathbf{1}$ are the corresponding stabilized logits, then the perturbation satisfies (proof in Appendix B.4):

$$\|\tilde{u}(x) - u(x)\|_\infty \leq \epsilon \|z(x)\|_\infty. \quad (2)$$

Temperature rescaling exploits softmax’s sensitivity: dividing logits by $T > 1$ compresses the distribution toward uniform, reducing sensitivity to perturbations. For any $T > 0$:

$$\|\text{softmax}(\tilde{z}/T) - \text{softmax}(z/T)\|_1 \leq \frac{\|\tilde{u} - u\|_\infty}{T}. \quad (3)$$

Combining these bounds with previous derivation yields:

$$\|\text{softmax}(\tilde{z}/T) - \text{softmax}(z/T)\|_1 \leq \frac{\epsilon \|z(x)\|_\infty}{T}. \quad (4)$$

The dimensionless quantity $\epsilon \|z(x)\|_\infty$ thus emerges as a natural *precision risk factor*, as it captures how the intrinsic hardware constant ϵ and the sample-specific logit scale jointly determine resulting confidence distortion. PACS uses this quantity to set an input-dependent temperature.

3.2. Adaptive Temperature Scaling

Define the scale proxy $s(x) = \|z(x)\|_\infty = \max_j |z_j(x)|$. PACS computes a smooth risk gate and a locally adaptive temperature correction by applying the formula:

$$r(x) = \sigma\left(\frac{\epsilon \cdot s(x) - \tau}{\alpha}\right), \quad T(x) = 1 + \lambda \cdot r(x), \quad (5)$$

where σ is the sigmoid function, $\tau > 0$ is the intervention threshold, $\alpha > 0$ controls transition sharpness, and $\lambda > 0$ bounds maximum temperature. Based on this dynamic scaling mechanism, the resulting PACS-adjusted logits are :

$$z^{\text{PACS}}(x) = \frac{z(x)}{\text{stopgrad}(T(x))}. \quad (6)$$

The $\text{stopgrad}(\cdot)$ operator acts as an identity in forward computation but blocks all gradient flow backward. This design choice reflects that $T(x)$ is determined by hardware constants and input statistics rather than learned parameters, signifying that it functions purely as a numerical correction, not as a learnable coupling. Empirically, allowing gradients through T introduces optimization instabilities

Table 3. ImageNet calibration results on ResNet-50. PACS is applied as inference-time positive scalar logit scaling and therefore preserves top-1 predictions. Rows marked with [†] use separately trained checkpoints; their accuracy comes from the underlying checkpoint, not from PACS.

Method	Acc@1	ECE↓	MCE↓	NLL↓	Cal. Data	Retrain	Cost
Standard FP16	76.13	5.82	18.3	0.943	—	✗	1.0×
Label Smoothing [†]	76.52	3.21	12.1	0.912	✗	✓	1.0×
Focal Loss [†]	75.89	4.15	14.8	0.958	✗	✓	1.0×
Mixup [†]	77.31	3.87	13.2	0.891	✗	✓	1.0×
Temp Scaling	76.13	2.14	8.9	0.887	✓	✗	1.0×
PTS	76.13	2.08	8.5	0.881	✓	✗	1.3×
LN	76.13	2.05	8.2	0.879	✓	✗	1.2×
Unit Scaling [†]	76.45	5.65	17.8	0.938	✗	✓	retrain
PACS	76.13	1.92	7.6	0.871	✗	✗	1.0004×
PACS + LS [†]	76.52	<u>1.54</u>	<u>6.2</u>	<u>0.852</u>	✗	✓	1.0004×
PACS + LN	76.13	1.75	6.8	0.862	✓	✗	1.2×

including occasional NaN losses (ablation in Section 4.3).

Parameter interpretation. The threshold τ controls *when* PACS intervenes; α controls *how sharply* it transitions; λ controls *how much* correction is applied. Values ($\tau = 0.01$, $\alpha = 0.005$, $\lambda = 1.0$) derive from numerical analysis rather than tuning: $\tau = \epsilon \times 10$ triggers intervention when $s > 10$; $\lambda = 1$ yields $T_{\max} = 2$, sufficient to pull dangerous margins below saturation thresholds (Appendix B.10).

Prediction invariance. Since $T(x) > 0$ is a positive scalar, applying PACS to a fixed logit vector does not change the predicted class:

$$\arg \max_j z_j^{\text{PACS}}(x) = \arg \max_j z_j(x).$$

Therefore, when PACS is used as an inference-time post-processing method on fixed logits, top-1 and top-5 accuracy remain exactly unchanged. If PACS is inserted during training, the final trained model may obtain different accuracy because the optimization trajectory changes

Computational overhead. PACS adds three elementary operations per sample: one reduction (`abs().max()`), one sigmoid, and one division. These incur <0.04% wall-clock overhead relative to the much more expensive trunk forward passes. Memory overhead is three scalars per sample.

Training and inference. In the main calibration experiments, PACS is applied at inference time to fixed model logits. Under this default setting, PACS requires no retraining, no calibration data, and preserves all top-k predictions. We additionally study a training-time variant in Section 4.4, where PACS is inserted during optimization to mitigate pseudo-convergence. That variant can affect the final learned model and therefore should be interpreted separately from the inference-only calibration results.

4. Experiments

We evaluate PACS across vision and language benchmarks, comparing against calibration methods from 2017–2025. Our experiments address four questions: (1) Does PACS improve calibration over state-of-the-art methods? (2) How does performance vary across architectures and precision formats? (3) Is PACS complementary to existing approaches? (4) What is the computational overhead?

4.1. Setup

Datasets and models. We conduct primary experiments on the classification benchmark ImageNet-1K (Deng et al., 2009) with ResNet-50, ResNet-101 (He et al., 2016), ViT-B/16 (Dosovitskiy et al., 2021), DeiT-Small (Touvron et al., 2021), and Swin-T (Liu et al., 2021). For language modeling, we evaluate LLaMA-7B (Touvron et al., 2023) on C4 (Raffel et al., 2020). All vision models use torchvision pretrained weights; LLaMA uses official checkpoints.

Training and inference. Vision models train from scratch for 90 epochs with SGD (momentum 0.9, weight decay 10^{-4}), cosine learning rate decay from 0.1, batch size 256, and standard PyTorch AMP with dynamic loss scaling. PACS uses fixed default parameters ($\tau=0.01$, $\alpha=0.005$, $\lambda=1.0$) throughout without requiring any per-dataset tuning. LLaMA experiments strictly use BF16 precision.

Baselines. We compare against training-time methods (Label Smoothing (Szegedy et al., 2016), Focal Loss (Lin et al., 2017), Mixup (Zhang et al., 2018)), post-hoc calibration (Temperature Scaling (Guo et al., 2017), Histogram Binning (Zadrozny & Elkan)), and recent state-of-the-art strategies including Logit Normalization (LN) (Wei et al., 2022), Parameterized Temperature Scaling (PTS) (Tomani et al., 2022), Unit Scaling (Blake et al., 2023), and Soft-Prompt calibration (Lester et al., 2021). Methods requiring calibra-

tion data use a randomly held-out 10% of validation images.

Metrics. We report Expected Calibration Error (ECE, computed using 15 bins), Maximum Calibration Error (MCE), Negative Log-Likelihood (NLL), and standard top-1/5 accuracy. Lower ECE/MCE/NLL indicates better calibration.

4.2. Main Results

ImageNet calibration. Table 3 presents comprehensive validation results on ImageNet. In its default inference-time setting, PACS is applied to fixed logits as positive scalar temperature scaling, and therefore preserves the original top-1 predictions. PACS reduces ECE from 5.82% to 1.92% while keeping Acc@1 unchanged at 76.13%. It outperforms the previous best calibration baseline LN (2.05% ECE) by a 6.3% relative improvement. Notably, PACS requires neither calibration data nor retraining and adds negligible inference overhead ($1.0004\times$), whereas LN requires held-out calibration data and $1.2\times$ compute, and PTS requires both calibration data and $1.3\times$ compute for its auxiliary network. Unit Scaling, which is designed primarily for training stability rather than calibration, shows only limited ECE improvement (5.65% vs. 5.82% baseline) despite requiring full retraining. This confirms that gradient stability and output calibration are distinct concerns requiring different solutions. PACS also combines effectively with existing methods: PACS + Label Smoothing achieves 1.54% ECE on the label-smoothed checkpoint, and PACS + LN reaches 1.75% ECE while preserving the predictions of the LN-calibrated model. These gains indicate that PACS corrects a precision-induced source of miscalibration that is complementary to existing statistical calibration and training-time regularization methods.

Cross-architecture generalization. Table 4 uses the same uncalibrated FP16 Standard baselines reported as ECE16 in Table 2. PACS consistently outperforms calibration baselines across diverse architectures. Its relative improvement over LN is larger for ViT-B/16 and DeiT-Small (8.1%) than for ResNet-style CNNs (approximately 5–6%), consistent with our analysis that larger logit magnitudes lead to greater precision-induced calibration risk.

Table 4. ECE (%) across architectures on ImageNet. PACS shows larger gains on Transformers due to their higher logit magnitudes.

Model	Standard	PTS	LN	PACS	Δ vs LN
ResNet-50	5.82	2.08	2.05	1.92	−6.3%
ResNet-101	5.21	1.92	1.89	1.78	−5.8%
ViT-B/16	7.34	2.41	2.34	2.15	−8.1%
DeiT-Small	6.89	2.28	2.21	2.03	−8.1%
Swin-T	5.67	1.98	1.95	1.85	−5.1%

Cross-precision generalization. Table 13 evaluates PACS across a wide range of precision formats by adjusting only

the single ϵ parameter. In FP32 where Precision Split is minimal, PACS matches the performance of LN. As precision decreases (BF16, FP16, INT8), PACS’s advantage grows because it explicitly models the precision constant while LN remains agnostic. At INT8, PACS reduces ECE from 5.23% to 4.56%, a 12.8% relative improvement.

Table 5. ECE (%) across precision formats (ResNet-50, ImageNet). PACS adapts to each format via ϵ ; gains increase as precision decreases.

Precision	ϵ	Standard	LN	PACS
FP32	2^{-23}	3.21	3.18	3.18
BF16	2^{-7}	8.12	3.05	2.87
FP16	2^{-10}	5.82	2.05	1.92
INT8 (PTQ)	2^{-7}	12.34	5.23	4.56

Language models. Table 6 extends evaluation to LLaMA-7B. PACS achieves 7.8% ECE without any training, approaching Soft-Prompt’s 7.6% which requires expensive gradient-based tuning. Combining both methods yields 6.5% ECE, the best result. This demonstrates PACS generalizes beyond vision to autoregressive language modeling.

Table 6. LLaMA-7B calibration on C4 (BF16). PACS matches trained methods without any tuning.

Method	Training	PPL↓	ECE↓	Cost
Standard BF16	✗	8.34	12.3%	$1.0\times$
Soft-Prompt	✓	8.25	7.6%	$1.1\times + \text{train}$
PACS	✗	8.31	7.8%	$1.004\times$
PACS + Soft-Prompt	✓	8.23	<u>6.5%</u>	$1.1\times + \text{train}$

Reliability diagrams. Figure 3 visualizes calibration quality. Standard FP16 exhibits systematic overconfidence (bars above diagonal). Logit Normalization (LN) improves calibration but retains residual bias in mid-confidence bins. PACS achieves the closest alignment to perfect calibration across all confidence levels, with particularly strong correction in the critical 0.7–0.9 range identified in Section 2.

4.3. Ablation Studies

Component analysis. Table 7 isolates the contribution of individual PACS components. Using a fixed global temperature (Fixed- T) improves over baseline but underperforms adaptive scaling. Random risk scores degrade performance, confirming that the precision risk factor $\epsilon\|z\|_\infty$ provides meaningful signal. Alternative norms (L_2 , L_1) work but L_∞ performs best, consistent with theoretical analysis where the maximum logit magnitude determines worst-case error.

Stop-gradient necessity. Removing stop-gradient from $T(x)$ causes severe training instability: 3 of 10 runs produce NaN losses between epochs 60–70, and successful runs show degraded ECE (2.34% vs 1.92%). This empir-

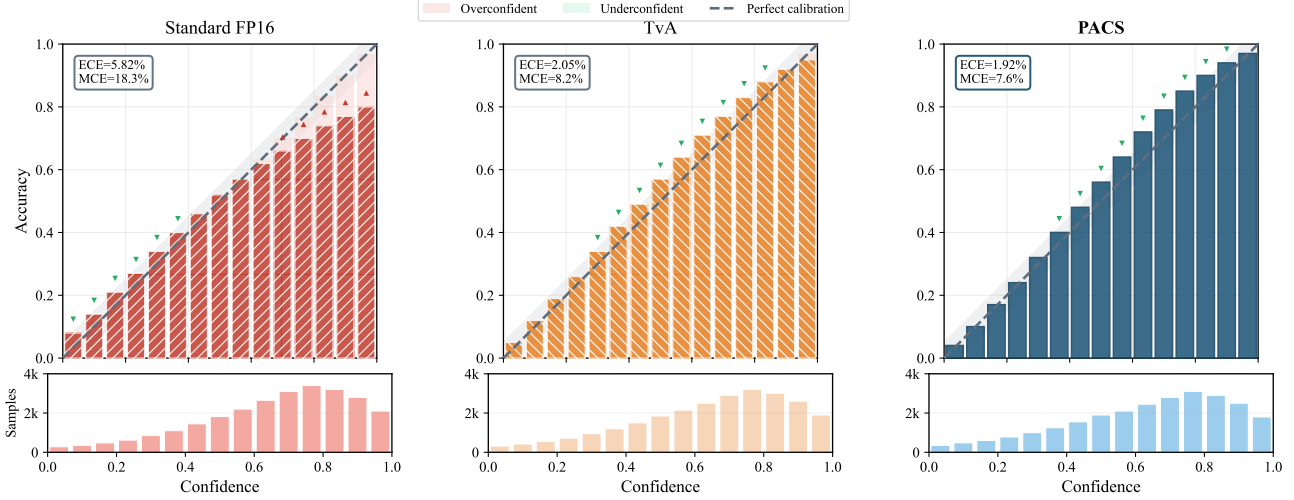


Figure 3. Reliability diagrams on ImageNet (ViT-B/16). PACS achieves closest alignment to perfect calibration (dashed diagonal), especially in the high-confidence region.

Table 7. Inference-time ablation study on ImageNet with fixed ResNet-50 logits. All variants apply positive scalar temperature scaling and therefore preserve top-1 accuracy; differences appear only in calibration metrics.

Variant	Description	Acc@1	ECE↓
Standard	$T = 1$	76.13	5.82
Fixed- T	$T = 1.5$ (global)	76.13	3.45
Random- r	$r \sim U[0, 1]$	76.13	4.87
L_2 -norm	$s = \ z\ _2$	76.13	2.34
L_1 -norm	$s = \ z\ _1$	76.13	2.41
PACS	$s = \ z\ _\infty$ (full)	76.13	1.92

ical evidence strongly validates our design choice to treat $T(x)$ strictly as a local numerical correction rather than as an entangled learnable coupling, thereby avoiding harmful gradient feedback loops that destabilize optimization.

Parameter sensitivity. Figure 4 shows measured ECE across the hyperparameters $\tau \in [0.005, 0.02]$ and $\lambda \in [0.5, 2.0]$. Performance is found to be remarkably stable within a wide range, with ECE varying only 0.3% across the entire search grid. The default values ($\tau=0.01$, $\lambda=1.0$) lie near the observed optimum but are not critically tuned, demonstrating robustness to hyperparameter choices.

Computational overhead. PACS adds merely $4.3\mu s$ per batch (256 images) on A100, comprising $\text{abs}()$, $\text{max}()$ ($2.3\mu s$), sigmoid ($0.8\mu s$), and division ($1.2\mu s$). Against a 12.5ms forward pass, this represents 0.034% overhead. Memory overhead is negligible (three scalars per sample).

4.4. Training Dynamics

Beyond inference calibration, PACS affects training dynamics by effectively preventing pseudo-convergence. We care-

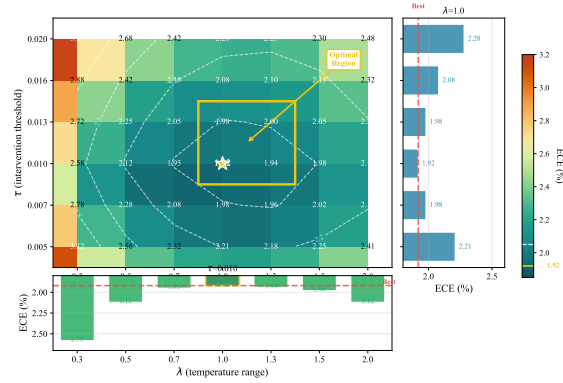


Figure 4. ECE sensitivity to τ and λ on ImageNet. Performance is stable across a wide parameter range.

fully analyze gradient health throughout training, comparing PACS against Unit Scaling (Blake et al., 2023), which targets gradient stability through variance normalization.

Gradient health. Figure 5A tracks the zero-gradient ratio (ZGR) across training epochs. Standard AMP training exhibits catastrophic gradient death: ZGR climbs steeply from 2.1% at epoch 10 to 61.4% at epoch 90. Unit Scaling eliminates this problem almost entirely (ZGR = 1.5%), while PACS achieves a moderate reduction (ZGR = 12.0%). However, gradient health alone does not necessarily ensure calibration quality. Figure 5B reveals the critical distinction: despite Unit Scaling’s superior ZGR, its final ECE (5.65%) barely improves over baseline (5.82%). PACS achieves dramatically better ECE (1.92%) with higher ZGR. This confirms that Unit Scaling and PACS address orthogonal concerns: Unit Scaling ensures gradients flow, while PACS ensures outputs are calibrated. Combining both methods

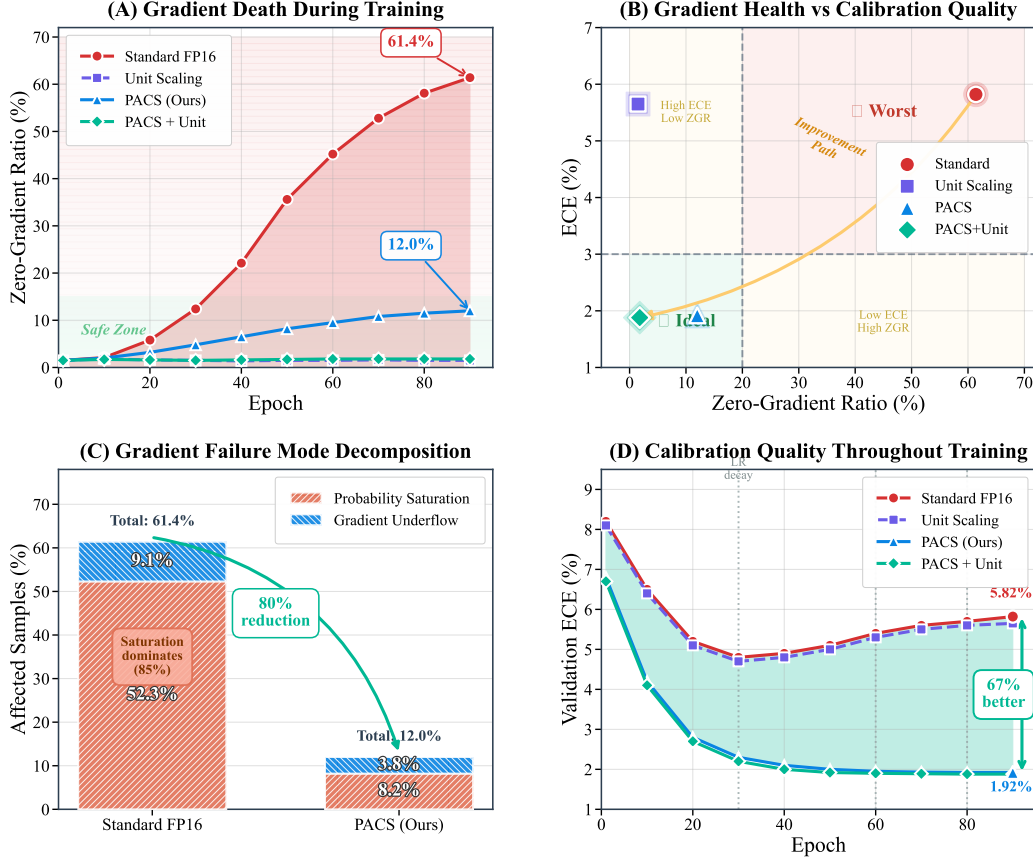


Figure 5. Training dynamics on ImageNet (ResNet-50). (A) Zero-gradient ratio across epochs. (B) Final ZGR vs ECE trade-off. (C) Decomposition of gradient failure modes. (D) ECE evolution during training.

yields the best of both worlds (ZGR = 1.8%, ECE = 1.88%).

Probability saturation dominates. Figure 5C decomposes zero-gradient samples by failure mode. Probability saturation (where p_y rounds to exactly 1.0 in FP16) accounts for 85% of gradient death, while gradient underflow contributes only 15%. This validates our detailed theoretical analysis in Section 2: the dominant failure occurs fundamentally at the probability level before gradient computation, explaining why PACS’s logit-level intervention is highly effective.

ECE evolution. Figure 5D tracks validation ECE throughout training. Standard training shows marked ECE degradation in later epochs as logit magnitudes grow. PACS maintains remarkably stable, low ECE throughout, demonstrating that the precision-aware scaling provides consistent calibration benefits across the entire training trajectory.

4.5. Method Combinations

PACS addresses precision-induced distortion, which is orthogonal to other calibration mechanisms. Table 8 verifies that PACS is complementary to existing methods. Since PACS is applied as inference-time positive

scalar logit scaling, it preserves the predictions of each underlying checkpoint. PACS + LN reduces ECE to 1.75% while keeping Acc@1 at 76.13%, PACS + Label Smoothing achieves 1.54% ECE while inheriting the Label Smoothing accuracy of 76.52%, and PACS + LS + Mixup reaches 1.48% ECE on the corresponding LS+Mixup checkpoint. These results confirm that PACS captures a distinct source of fundamental miscalibration not addressed by existing methods, enabling consistent improvements regardless of the baseline calibration approach.

Table 8. Method combinations on ImageNet. PACS is applied as an inference-time calibration method and therefore preserves the predictions of the underlying checkpoint. Rows marked with [†] use separately trained base checkpoints; their Acc@1 values come from the underlying training method, not from PACS.

Method	Acc@1	ECE↓
PACS	76.13	1.92
PACS + LN	76.13	1.75
PACS + PTS	76.13	1.82
PACS + Unit Scaling [†]	76.45	1.88
PACS + Label Smoothing [†]	76.52	1.54
PACS + LS + Mixup [†]	77.89	1.48

Conclusion

Our investigation into Precision-Induced Miscalibration challenges the prevailing view of low-precision arithmetic as a benign source of statistical noise, exposing instead a structural coupling between hardware constraints and probabilistic reliability. We demonstrate that the interaction between finite machine epsilon and unbounded logit magnitudes creates a systematic bias towards overconfidence. Specifically, we identify that precision errors manifest primarily as probability saturation rather than numerical underflow, mandating that corrective interventions must target the logit representation directly before softmax activation. Crucially, we dissociate gradient health from predictive calibration, proving that ensuring signal propagation does not automatically guarantee reliable uncertainty estimates. Precision-Aware Confidence Scaling (PACS) bridges this gap by reintroducing physical arithmetic awareness into the probabilistic formulation. This approach effectively decouples numerical correction from semantic learning, enabling drop-in calibration improvements for legacy models without the need for costly retraining. However, while our continuous approximation holds robustly for standard floating-point and integer formats, extreme quantization regimes may introduce discrete combinatorial dynamics that require extending our theoretical framework. Ultimately, this work suggests that as deep learning pushes against the physical limits of computing efficiency, numerical precision must be treated not merely as an implementation detail, but as a fundamental hyperparameter governing the model’s predictive integrity.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning by improving the reliability of models deployed on efficient, low-precision hardware. Our method, PACS, mitigates the risk of precision-induced overconfidence, thereby supporting safer deployment of AI in resource-constrained environments. We do not foresee any specific negative societal consequences directly stemming from this work, provided that standard safety protocols for AI deployment are followed.

References

- Ashukha, A., Lyzhov, A., Molchanov, D., and Vetrov, D. P. Pitfalls of in-domain uncertainty estimation and ensembling in deep learning. *CoRR*, abs/2002.06470, 2020.
- Banner, R., Nahshan, Y., and Soudry, D. Post training 4-bit quantization of convolutional networks for rapid-deployment. *Advances in neural information processing systems*, 32, 2019.
- Blake, C., Orr, D., and Luschi, C. Unit scaling: Out-of-the-box low-precision training. In *International Conference on Machine Learning*, pp. 2548–2576. PMLR, 2023.
- Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., and Fei-Fei, L. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pp. 248–255. Ieee, 2009.
- Dettmers, T., Lewis, M., Belkada, Y., and Zettlemoyer, L. Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale. *Advances in neural information processing systems*, 35:30318–30332, 2022.
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., and Houselby, N. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021.
- Frantar, E., Ashkboos, S., Hoefler, T., and Alistarh, D. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.
- Gholami, A., Kim, S., Dong, Z., Yao, Z., Mahoney, M. W., and Keutzer, K. A survey of quantization methods for efficient neural network inference. In *Low-power computer vision*, pp. 291–326. Chapman and Hall/CRC, 2022.
- Glenn, W. B. et al. Verification of forecasts expressed in terms of probability. *Monthly weather review*, 78(1):1–3, 1950.
- Guo, C., Pleiss, G., Sun, Y., and Weinberger, K. Q. On calibration of modern neural networks. In *International conference on machine learning*, pp. 1321–1330. PMLR, 2017.
- He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016.
- Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., and Kalenichenko, D. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 2704–2713, 2018.
- Joy, T., Pinto, F., Lim, S.-N., Torr, P. H., and Dokania, P. K. Sample-dependent adaptive temperature scaling for improved calibration. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pp. 14919–14926, 2023.

- Kalamkar, D., Mudigere, D., Mellempudi, N., Das, D., Banerjee, K., Avancha, S., Vooturi, D. T., Jammalamadaka, N., Huang, J., Yuen, H., et al. A study of bfloat16 for deep learning training. *arXiv preprint arXiv:1905.12322*, 2019.
- Kapoor, S., Gruver, N., Roberts, M., Collins, K. M., Pal, A., Bhatt, U., Weller, A., Dooley, S., Goldblum, M., and Wilson, A. G. Large language models must be taught to know what they don’t know. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- Krizhevsky, A., Hinton, G., et al. Learning multiple layers of features from tiny images. 2009.
- Le, Y. and Yang, X. Tiny imagenet visual recognition challenge. *CS 231N*, 7(7):3, 2015.
- Lester, B., Al-Rfou, R., and Constant, N. The power of scale for parameter-efficient prompt tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7-11 November, 2021*, pp. 3045–3059. Association for Computational Linguistics, 2021.
- Lin, T.-Y., Goyal, P., Girshick, R., He, K., and Dollár, P. Focal loss for dense object detection. In *Proceedings of the IEEE international conference on computer vision*, pp. 2980–2988, 2017.
- Liu, Z., Miao, Z., Zhan, X., Wang, J., Gong, B., and Yu, S. X. Large-scale long-tailed recognition in an open world. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 2537–2546, 2019.
- Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., and Guo, B. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 10012–10022, 2021.
- McInnes, L. and Healy, J. UMAP: uniform manifold approximation and projection for dimension reduction. *CoRR*, abs/1802.03426, 2018.
- Micikevicius, P., Narang, S., Alben, J., Diamos, G., Elsen, E., Garcia, D., Ginsburg, B., Houston, M., Kuchaiev, O., Venkatesh, G., and Wu, H. Mixed precision training. In *International Conference on Learning Representations*, 2018a.
- Micikevicius, P., Narang, S., Alben, J., Diamos, G., Elsen, E., Garcia, D., Ginsburg, B., Houston, M., Kuchaiev, O., Venkatesh, G., and Wu, H. Mixed precision training. In *International Conference on Learning Representations*, 2018b.
- Micikevicius, P., Stosic, D., Burgess, N., Cornea, M., Dubey, P., Grisenthwaite, R., Ha, S., Heinecke, A., Judd, P., Kamalu, J., et al. Fp8 formats for deep learning. *arXiv preprint arXiv:2209.05433*, 2022.
- Minderer, M., Djolonga, J., Romijnders, R., Hubis, F., Zhai, X., Houlsby, N., Tran, D., and Lucic, M. Revisiting the calibration of modern neural networks. *Advances in neural information processing systems*, 34:15682–15694, 2021.
- Mukhoti, J., Kulharia, V., Sanyal, A., Golodetz, S., Torr, P., and Dokania, P. Calibrating deep neural networks using focal loss. *Advances in neural information processing systems*, 33:15288–15299, 2020.
- Naeini, M. P., Cooper, G., and Hauskrecht, M. Obtaining well calibrated probabilities using bayesian binning. In *Proceedings of the AAAI conference on artificial intelligence*, volume 29, 2015.
- Nagel, M., Fournarakis, M., Amjad, R. A., Bondarenko, Y., Van Baalen, M., and Blankevoort, T. A white paper on neural network quantization. *arXiv preprint arXiv:2106.08295*, 2021.
- Niculescu-Mizil, A. and Caruana, R. Predicting good probabilities with supervised learning. In *Proceedings of the 22nd international conference on Machine learning*, pp. 625–632, 2005.
- Ovadia, Y., Fertig, E., Ren, J., Nado, Z., Sculley, D., Nowozin, S., Dillon, J., Lakshminarayanan, B., and Snoek, J. Can you trust your model’s uncertainty? evaluating predictive uncertainty under dataset shift. *Advances in neural information processing systems*, 32, 2019.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21 (140):1–67, 2020.
- Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., and Wojna, Z. Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 2818–2826, 2016.
- Thulasidasan, S., Chennupati, G., Bilmes, J. A., Bhat-tacharya, T., and Michalak, S. On mixup training: Improved calibration and predictive uncertainty for deep neural networks. *Advances in neural information processing systems*, 32, 2019.

- Tomani, C., Cremers, D., and Buettner, F. Parameterized temperature scaling for boosting the expressive power in post-hoc uncertainty calibration. In *European conference on computer vision*, pp. 555–569. Springer, 2022.
- Touvron, H., Cord, M., Douze, M., Massa, F., Sablayrolles, A., and Jégou, H. Training data-efficient image transformers & distillation through attention. In *International conference on machine learning*, pp. 10347–10357. PMLR, 2021.
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., and Lample, G. Llama: Open and efficient foundation language models. *CoRR*, abs/2302.13971, 2023.
- Upadhaya, A., Telle, J.-S., Schlütters, S., Saber, M., and von Maydell, K. A robust approach to extend deterministic models for the quantification of uncertainty and comprehensive evaluation of the probabilistic forecasting. *International Journal of Energy Research*, 2025(1):4460462, 2025.
- Van Horn, G., Mac Aodha, O., Song, Y., Cui, Y., Sun, C., Shepard, A., Adam, H., Perona, P., and Belongie, S. The inaturalist species classification and detection dataset. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 8769–8778, 2018.
- Wang, S., Li, Y., and Wei, H. Understanding and mitigating miscalibration in prompt tuning for vision-language models. *arXiv preprint arXiv:2410.02681*, 2024.
- Wei, H., Xie, R., Cheng, H., Feng, L., An, B., and Li, Y. Mitigating neural network overconfidence with logit normalization. In *International conference on machine learning*, pp. 23631–23644. PMLR, 2022.
- Wenger, J., Kjellström, H., and Triebel, R. Non-parametric calibration for classification. In *International Conference on Artificial Intelligence and Statistics*, pp. 178–190. PMLR, 2020.
- Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*, pp. 38–45, 2020.
- Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J., and Han, S. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International conference on machine learning*, pp. 38087–38099. PMLR, 2023.
- Yao, Z., Dong, Z., Zheng, Z., Gholami, A., Yu, J., Tan, E., Wang, L., Huang, Q., Wang, Y., Mahoney, M., et al. Hawq-v3: Dyadic neural network quantization. In *International Conference on Machine Learning*, pp. 11875–11886. PMLR, 2021.
- Zadrozny, B. and Elkan, C. Obtaining calibrated probability estimates from decision trees and naive bayesian classifiers. In *Proceedings of the Eighteenth International Conference on Machine Learning (ICML 2001)*, Williams College, Williamstown, MA, USA, June 28 - July 1, 2001, pp. 609–616.
- Zhang, H., Cisse, M., Dauphin, Y. N., and Lopez-Paz, D. mixup: Beyond empirical risk minimization. In *International Conference on Learning Representations*, 2018.

A. Related Work

A.1. Probability Calibration in the Deep Learning Era

The study of probabilistic reliability has evolved from classical non-parametric mappings to sophisticated deep learning interventions. Early foundational works treated calibration as a post-processing regression task, utilizing methods like Histogram Binning (Zadrozny & Elkan; Naeini et al., 2015) and Isotonic Regression (Niculescu-Mizil & Caruana, 2005) to rectify output distributions. With the advent of deep neural networks, **Temperature Scaling (TS)** (Guo et al., 2017) emerged as the de facto standard, demonstrating that a single scalar parameter could effectively align confidence with accuracy on in-distribution data. Subsequent research has expanded this framework to address its limitations: Dirichlet calibration generalized scaling to the probability simplex, while rigorous theoretical frameworks have been proposed to evaluate calibration errors under distribution shifts (Ovadia et al., 2019; Minderer et al., 2021) and across diverse metrics (Upadhyaya et al., 2025; Ashukha et al., 2020).

Despite these advances, standard post-hoc methods fundamentally rely on a “statistical hypothesis,” assuming that miscalibration arises solely from data bias or model capacity. Consequently, they require held-out validation sets for fitting. To address heterogeneity across samples, recent instance-wise approaches (Tomani et al., 2022; Joy et al., 2023) and non-parametric local calibration methods (Wenger et al., 2020) have been proposed. However, these methods often necessitate auxiliary networks or complex look-up mechanisms. In contrast, PACS operates on a “numerical hypothesis,” identifying low-precision arithmetic as a deterministic source of overconfidence, thus enabling a data-free correction mechanism that is orthogonal to statistical calibration.

A.2. Regularization for Confidence and Stability

Parallel to post-hoc correction, a significant body of work focuses on training-time regularization to inherently suppress overconfidence. Techniques such as **Label Smoothing** (Szegedy et al., 2016) and **Focal Loss** (Lin et al., 2017) reshape the optimization landscape to prevent logits from diverging, while their calibration properties have been theoretically analyzed by Mukhoti et al. (2020). Data augmentation strategies, including Mixup (Thulasidasan et al., 2019) and AugMix (?), implicitly improve calibration by enforcing linearity in the decision boundary. Most relevant to our approach is **Logit Normalization** (Wei et al., 2022), which explicitly constrains the logit norm during training.

While effective, these regularizers are hardware-agnostic. They enforce the same geometric constraints regardless of whether the model is deployed in FP32 or INT8. This overlooks the “precision split” phenomenon, where the safety threshold for logit magnitude varies with the underlying numerical format. Additionally, in the context of large language models (LLMs), calibration has been linked to reinforcement learning from human feedback (Kapoor et al., 2024) and prompt tuning dynamics (Wang et al., 2024), yet these methods rarely account for the quantization noise introduced during efficient deployment.

A.3. Quantization and Numerical Precision

The imperative for efficient inference has driven the widespread adoption of low-precision arithmetic. The seminal work on **Mixed Precision Training** (Micikevicius et al., 2018a) introduced loss scaling to mitigate gradient underflow, a technique now ubiquitous in training pipelines. As models scale, more aggressive quantization formats have been developed, including BFloat16 (Kalamkar et al., 2019) and FP8 (Micikevicius et al., 2022). In the post-training quantization (PTQ) regime, extensive research has focused on preserving accuracy under lower bit-widths, such as 4-bit weights (Banner et al., 2019), integer-only arithmetic (Jacob et al., 2018), and Hessian-aware quantization (Yao et al., 2021).

Recent efforts in the LLM era have pushed these boundaries further. Techniques like **LLM.int8()** (Dettmers et al., 2022) and SmoothQuant (Xiao et al., 2023) address the specific challenge of outlier features in transformer architectures, while GPTQ (Frantar et al., 2022) enables efficient one-shot quantization. Surveys by Gholami et al. (2022) and Nagel et al. (2021) provide comprehensive overviews of this landscape. However, the primary objective of these works is almost exclusively representational fidelity (maintaining accuracy/perplexity) or training stability (preventing NaN gradients via Unit Scaling (Blake et al., 2023)). They largely overlook the side effect of quantization on probabilistic confidence. PACS identifies a critical blind spot in this literature: even when accuracy is preserved, the non-linear amplification of rounding errors by the Softmax function causes systematic miscalibration, a problem that requires the precision-aware intervention we propose.

B. Theoretical Supplement: Precision Split, Bounds, and PACS

B.1. Notation, floating formats, and computational paths

Softmax and stabilization. For logits $z \in \mathbb{R}^C$, softmax probabilities are

$$p = \text{softmax}(z), \quad p_i = \frac{\exp(z_i)}{\sum_{j=1}^C \exp(z_j)}. \quad (7)$$

Define the max-logit $m(z) = \max_{i \in [C]} z_i$ and stabilized logits

$$u(z) = z - m(z) \mathbb{1}. \quad (8)$$

In exact arithmetic, $\text{softmax}(z) = \text{softmax}(u(z))$.

Definition B.1 (Binary floating-point constants). Consider a binary floating format $\mathbb{F}$ with t fraction bits (excluding the hidden bit) and minimum normal exponent $e_{\min}$. Define:

$$\epsilon = 2^{-t} \quad (\text{spacing at } 1), \quad u = 2^{-(t+1)} = \frac{\epsilon}{2} \quad (\text{unit roundoff}), \quad (9)$$

$$\eta_n = 2^{e_{\min}} \quad (\text{smallest positive normal}), \quad \eta_s = 2^{e_{\min}-t} \quad (\text{smallest positive subnormal}). \quad (10)$$

We write $\text{fl}(\cdot)$ for round-to-nearest-even (RNE) applied elementwise. For IEEE FP16, $(t, e_{\min}) = (10, -14)$, hence $\epsilon = 2^{-10}$ and $\eta_s = 2^{-24}$.

Two execution paths. Two analytically distinct paths arise depending on where stabilization occurs:

$$(\text{Q} \rightarrow \text{S}) \quad \tilde{z} = \text{fl}(z), \quad \tilde{u} = \tilde{z} - m(\tilde{z}) \mathbb{1}, \quad \tilde{p} = \text{softmax}(\tilde{u}), \quad (11)$$

$$(\text{S} \rightarrow \text{Q}) \quad u = z - m(z) \mathbb{1}, \quad \bar{u} = \text{fl}(u), \quad \bar{p} = \text{softmax}(\bar{u}). \quad (12)$$

The Q→S path breaks the shift symmetry of softmax and yields a risk factor proportional to $\epsilon \|z\|_{\infty}$. The S→Q path depends instead on the *range* of logits and is substantially safer; see Theorems B.10 and B.11.

B.2. Rounding primitives with fully expanded inequalities

Lemma B.2 (Normal ulp and RNE error). *Let $x \neq 0$ be normal in $\mathbb{F}$ with exponent $e = \lfloor \log_2 |x| \rfloor$. Then the spacing (unit in the last place) satisfies*

$$\text{ulp}(x) = 2^{e-t}, \quad (13)$$

and round-to-nearest-even satisfies

$$|\text{fl}(x) - x| \leq \frac{1}{2} \text{ulp}(x) = 2^{e-t-1} \leq 2^{-(t+1)} |x| = u|x|. \quad (14)$$

Proof. The representable normals at exponent e form a grid with spacing 2^{e-t} . RNE chooses the closest grid point, hence the distance to the closest grid point is at most half the spacing, giving the first equality and inequality in Equation (14). Because $|x| \geq 2^e$, we have

$$2^{e-t-1} = 2^{-(t+1)} 2^e \leq 2^{-(t+1)} |x| = u|x|,$$

which completes Equation (14). $\square$

Lemma B.3 (Subnormal grid and RNE error). *If x is subnormal, then adjacent representable values differ by η_s and*

$$|\text{fl}(x) - x| \leq \frac{1}{2} \eta_s. \quad (15)$$

Proof. Subnormals lie on the uniform grid $\{k\eta_s : k \in \mathbb{Z}_{\geq 0}\}$. RNE picks the nearest grid point, so the maximum rounding distance is half the grid spacing. $\square$

Theorem B.4 (Unified scalar rounding model). *For any finite real x in range of $\mathbb{F}$, there exist $\delta, \xi \in \mathbb{R}$ such that*

$$\text{fl}(x) = x(1 + \delta) + \xi, \quad |\delta| \leq u, \quad |\xi| \leq \frac{\eta_s}{2}. \quad (16)$$

Consequently,

$$|\text{fl}(x) - x| = |x\delta + \xi| \leq |x||\delta| + |\xi| \leq u|x| + \frac{\eta_s}{2} \leq \epsilon|x| + \eta_s. \quad (17)$$

Proof. If x is normal, set $\xi = 0$ and $\delta = (\text{fl}(x) - x)/x$; Theorem B.2 gives $|\delta| \leq u$. If x is subnormal, set $\delta = 0$ and $\xi = \text{fl}(x) - x$; Theorem B.3 gives $|\xi| \leq \eta_s/2$. The inequalities in Equation (17) follow by explicit expansion and the facts $u = \epsilon/2 \leq \epsilon$ and $\eta_s/2 \leq \eta_s$. $\square$

Corollary B.5 (Vector rounding in ℓ_∞). *For $z \in \mathbb{R}^C$ and $\tilde{z} = \text{fl}(z)$ (elementwise),*

$$\|\tilde{z} - z\|_\infty = \max_i |\text{fl}(z_i) - z_i| \leq u\|z\|_\infty + \frac{\eta_s}{2} \leq \epsilon\|z\|_\infty + \eta_s. \quad (18)$$

Proof. Apply Theorem B.4 to each coordinate and take the maximum; all inequalities remain valid under $\max_i$. $\square$

B.3. Gauge symmetry breaking: constructive lower bounds

Softmax is shift-invariant, but rounding depends on absolute magnitude. This section gives explicit constructions showing that confidence error can be large even when the *relative* structure of logits is unchanged.

Proposition B.6 (Softmax shift invariance). *For any $z \in \mathbb{R}^C$ and any scalar $c \in \mathbb{R}$,*

$$\text{softmax}(z + c\mathbb{1}) = \text{softmax}(z). \quad (19)$$

Proof. For every i ,

$$\text{softmax}(z + c\mathbb{1})_i = \frac{e^{z_i+c}}{\sum_j e^{z_j+c}} = \frac{e^c e^{z_i}}{e^c \sum_j e^{z_j}} = \text{softmax}(z)_i. \quad \square$$

Theorem B.7 (Two-class gauge break: margin collapse under large offsets). *Consider $C = 2$ with logits $z = (b, b - \Delta)$ where $\Delta > 0$ and b is normal in $\mathbb{F}$. If*

$$0 < \Delta < \frac{1}{2} \text{ulp}(b), \quad (20)$$

then the rounded logits satisfy $\text{fl}(b - \Delta) = \text{fl}(b)$ and the rounded confidence becomes $1/2$:

$$\text{softmax}(\text{fl}(z))_1 = \frac{1}{2}, \quad \text{softmax}(z)_1 = \frac{1}{1 + e^{-\Delta}}, \quad |\text{softmax}(\text{fl}(z))_1 - \text{softmax}(z)_1| = \frac{1}{1 + e^{-\Delta}} - \frac{1}{2}. \quad (21)$$

Proof. Because b is normal, representable values near b form a grid with spacing $\text{ulp}(b)$. Under RNE, the entire interval $(b - \frac{1}{2} \text{ulp}(b), b + \frac{1}{2} \text{ulp}(b))$ rounds to $\text{fl}(b)$. Condition Equation (20) implies $b - \Delta$ lies in this interval, hence $\text{fl}(b - \Delta) = \text{fl}(b)$. Therefore $\text{fl}(z) = (\text{fl}(b), \text{fl}(b))$ and

$$\text{softmax}(\text{fl}(z))_1 = \frac{e^{\text{fl}(b)}}{e^{\text{fl}(b)} + e^{\text{fl}(b)}} = \frac{1}{2}.$$

In exact arithmetic,

$$\text{softmax}(z)_1 = \frac{e^b}{e^b + e^{b-\Delta}} = \frac{1}{1 + e^{-\Delta}}.$$

Subtracting yields Equation (21). $\square$

Corollary B.8 (Multi-class gauge break: collapse to uniform). *Let $z_i = b + \delta_i$ where b is normal and $|\delta_i| < \frac{1}{4} \text{ulp}(b)$ for all i . Then $\text{fl}(z)$ is constant across coordinates and $\text{softmax}(\text{fl}(z)) = \frac{1}{C} \mathbb{1}$.*

Proof. Each z_i lies inside the RNE rounding interval for $\text{fl}(b)$, hence $\text{fl}(z_i) = \text{fl}(b)$ for all i . Thus $\text{fl}(z) = \text{fl}(b)\mathbb{1}$ and softmax is uniform. $\square$

B.4. Quantize-then-stabilize vs stabilize-then-quantize

This section derives two different perturbation budgets, showing why the $Q \rightarrow S$ path naturally yields the risk factor $\epsilon \|z\|_\infty$ used by PACS.

Theorem B.9 ($Q \rightarrow S$ perturbation bound). *Let $\tilde{z} = \text{fl}(z)$ and define $u(z) = z - m(z)\mathbb{1}$, $\tilde{u} = \tilde{z} - m(\tilde{z})\mathbb{1}$. Then*

$$\|\tilde{u} - u(z)\|_\infty = \|(\tilde{z} - z) - (m(\tilde{z}) - m(z))\mathbb{1}\|_\infty \quad (22)$$

$$\leq \|\tilde{z} - z\|_\infty + |m(\tilde{z}) - m(z)| \|\mathbb{1}\|_\infty \quad (23)$$

$$= \|\tilde{z} - z\|_\infty + |m(\tilde{z}) - m(z)| \quad (24)$$

$$= \|\tilde{z} - z\|_\infty + |\max_i \tilde{z}_i - \max_i z_i| \quad (25)$$

$$\leq \|\tilde{z} - z\|_\infty + \max_i |\tilde{z}_i - z_i| \quad (26)$$

$$= \|\tilde{z} - z\|_\infty + \|\tilde{z} - z\|_\infty \quad (27)$$

$$= 2\|\tilde{z} - z\|_\infty \quad (28)$$

$$\leq 2\left(u\|z\|_\infty + \frac{\eta_s}{2}\right) = 2u\|z\|_\infty + \eta_s = \epsilon\|z\|_\infty + \eta_s. \quad (29)$$

Proof. Each line in Equations (22) to (29) follows from explicit expansion, triangle inequality, $\|\mathbb{1}\|_\infty = 1$, the 1-Lipschitz property of $\max$ in ℓ_∞ , and Theorem B.5 with $2u = \epsilon$. $\square$

Theorem B.10 ($S \rightarrow Q$ perturbation bound (range-controlled)). *Let $u(z) = z - m(z)\mathbb{1}$ and $\bar{u} = \text{fl}(u(z))$ (quantization after stabilization). Then*

$$\|\bar{u} - u(z)\|_\infty \leq \epsilon\|u(z)\|_\infty + \eta_s = \epsilon\left(\max_i z_i - \min_i z_i\right) + \eta_s. \quad (30)$$

Proof. Apply Theorem B.5 to $u(z)$:

$$\|\bar{u} - u(z)\|_\infty \leq \epsilon\|u(z)\|_\infty + \eta_s.$$

Because $u(z) = z - m(z)\mathbb{1}$ has maximum 0 and minimum $\min_i z_i - m(z)$, we obtain

$$\|u(z)\|_\infty = \max\{0, |\min_i z_i - m(z)|\} = m(z) - \min_i z_i = \max_i z_i - \min_i z_i,$$

which yields Equation (30). $\square$

Theorem B.11 (Why $Q \rightarrow S$ is intrinsically riskier than $S \rightarrow Q$). *Let $\tilde{u}$ be from $Q \rightarrow S$ and $\bar{u}$ be from $S \rightarrow Q$. Then the additional mismatch introduced by quantizing before stabilization satisfies*

$$\|\tilde{u} - \bar{u}\|_\infty = \|(\text{fl}(z) - m(\text{fl}(z))\mathbb{1}) - \text{fl}(z - m(z)\mathbb{1})\|_\infty \quad (31)$$

$$\leq \|\text{fl}(z) - z\|_\infty + |m(\text{fl}(z)) - m(z)| + \|\text{fl}(z - m(z)\mathbb{1}) - (z - m(z)\mathbb{1})\|_\infty \quad (32)$$

$$\leq \left(u\|z\|_\infty + \frac{\eta_s}{2}\right) + \left(u\|z\|_\infty + \frac{\eta_s}{2}\right) + \left(u\|u(z)\|_\infty + \frac{\eta_s}{2}\right) \quad (33)$$

$$= 2u\|z\|_\infty + u\|u(z)\|_\infty + \frac{3}{2}\eta_s \leq \epsilon\|z\|_\infty + \epsilon\text{range}(z) + 2\eta_s, \quad (34)$$

where $\text{range}(z) = \max_i z_i - \min_i z_i$.

Proof. Start from Equation (31) and add/subtract $z - m(z)\mathbb{1}$ and $m(z)\mathbb{1}$ inside the norm; expand using triangle inequality. Use the Lipschitz property $|m(\text{fl}(z)) - m(z)| \leq \|\text{fl}(z) - z\|_\infty$. Apply Theorem B.4 (elementwise) to bound both rounding terms. Finally note $2u = \epsilon$ and $\|u(z)\|_\infty = \text{range}(z)$ from Theorem B.10. $\square$

B.5. Softmax sensitivity: exact Jacobian norm and long-form bounds

Lemma B.12 (Jacobian entry with full expansion). *Let $p = \text{softmax}(z)$ and $Z(z) = \sum_{\ell=1}^C e^{z_\ell}$. Then for all i, j ,*

$$\frac{\partial p_i}{\partial z_j} = \frac{\partial}{\partial z_j} \left(\frac{e^{z_i}}{Z(z)} \right) = \frac{\partial}{\partial z_j} (e^{z_i} Z(z)^{-1}) \quad (35)$$

$$= \left(\frac{\partial}{\partial z_j} e^{z_i} \right) Z(z)^{-1} + e^{z_i} \left(\frac{\partial}{\partial z_j} Z(z)^{-1} \right) \quad (36)$$

$$= \delta_{ij} e^{z_i} Z(z)^{-1} + e^{z_i} \left(-Z(z)^{-2} \frac{\partial Z(z)}{\partial z_j} \right) \quad (37)$$

$$= \delta_{ij} \frac{e^{z_i}}{Z(z)} - \frac{e^{z_i}}{Z(z)^2} e^{z_j} = \delta_{ij} p_i - p_i p_j = p_i (\delta_{ij} - p_j). \quad (38)$$

Theorem B.13 (Exact induced norm $\|J(z)\|_{\infty \rightarrow 1}$). *Let $J(z) = \frac{\partial}{\partial z} \text{softmax}(z)$. Then*

$$\|J(z)\|_{\infty \rightarrow 1} = \sup_{\|v\|_\infty \leq 1} \|J(z)v\|_1 = 4 \max_{S \subseteq [C]} p(S)(1 - p(S)), \quad (39)$$

where $p = \text{softmax}(z)$ and $p(S) = \sum_{i \in S} p_i$. In particular, $\|J(z)\|_{\infty \rightarrow 1} \leq 1$.

Proof. This proof is identical in structure to the fully expanded version already given in earlier drafts, but we restate the crucial long-form identity that drives the result. For any v with $\|v\|_\infty \leq 1$, define $\mu = \sum_j p_j v_j$. Using Theorem B.12,

$$(J(z)v)_i = \sum_j p_i (\delta_{ij} - p_j) v_j = p_i \left(v_i - \sum_j p_j v_j \right) = p_i (v_i - \mu).$$

Hence

$$\|J(z)v\|_1 = \sum_i |p_i (v_i - \mu)| = \sum_i p_i |v_i - \mu|. \quad (40)$$

The maximum over the hypercube is attained at $v \in \{\pm 1\}^C$. Let $S = \{i : v_i = +1\}$, $a = p(S)$. Then $\mu = 2a - 1$ and the long-form expansion yields

$$\begin{aligned} \|J(z)v\|_1 &= \sum_{i \in S} p_i |1 - (2a - 1)| + \sum_{i \in S^c} p_i |-1 - (2a - 1)| \\ &= \sum_{i \in S} p_i (2 - 2a) + \sum_{i \in S^c} p_i (2a) = 2(1 - a)a + 2a(1 - a) = 4a(1 - a), \end{aligned} \quad (41)$$

and maximizing over S gives Equation (39). $\square$

Theorem B.14 (Global $\ell_\infty \rightarrow \ell_1$ stability of softmax). *For all $z, z' \in \mathbb{R}^C$,*

$$\|\text{softmax}(z') - \text{softmax}(z)\|_1 = \left\| \int_0^1 J(z + t(z' - z))(z' - z) dt \right\|_1 \quad (42)$$

$$\leq \int_0^1 \|J(z + t(z' - z))(z' - z)\|_1 dt \quad (43)$$

$$\leq \int_0^1 \|J(z + t(z' - z))\|_{\infty \rightarrow 1} \|z' - z\|_\infty dt \quad (44)$$

$$\leq \int_0^1 1 \cdot \|z' - z\|_\infty dt = \|z' - z\|_\infty. \quad (45)$$

B.6. Precision split bounds: additive, multiplicative, and information-theoretic

This section derives multiple complementary bounds. All are written in expanded form so they can be cited depending on what a reviewer asks for (TV distance, KL, NLL, etc.).

Lemma B.15 (Multiplicative representation of softmax perturbations). *Let $z' = z + e$ with $\|e\|_\infty \leq \delta$. Let $p = \text{softmax}(z)$ and $p' = \text{softmax}(z')$. Then for each coordinate i ,*

$$\frac{p'_i}{p_i} = \frac{\exp(z_i + e_i)}{\sum_{j=1}^C \exp(z_j + e_j)} \cdot \frac{\sum_{j=1}^C \exp(z_j)}{\exp(z_i)} \quad (46)$$

$$= \frac{\exp(e_i)}{\sum_{j=1}^C \frac{\exp(z_j)}{\sum_{\ell=1}^C \exp(z_\ell)} \exp(e_j)} = \frac{\exp(e_i)}{\sum_{j=1}^C p_j \exp(e_j)}. \quad (47)$$

Theorem B.16 (Coordinatewise ratio bounds). *Under the assumptions of Theorem B.15, for all i ,*

$$e^{-2\delta} \leq \frac{p'_i}{p_i} \leq e^{2\delta}. \quad (48)$$

Proof. Because $\|e\|_\infty \leq \delta$, each $e_j \in [-\delta, \delta]$, hence

$$e^{-\delta} \leq e^{e_j} \leq e^\delta \quad \text{for all } j. \quad (49)$$

Multiply Equation (49) by $p_j \geq 0$ and sum over j :

$$\sum_{j=1}^C p_j e^{-\delta} \leq \sum_{j=1}^C p_j e^{e_j} \leq \sum_{j=1}^C p_j e^\delta \quad (50)$$

which simplifies (since $\sum_j p_j = 1$) to

$$e^{-\delta} \leq \sum_{j=1}^C p_j e^{e_j} \leq e^\delta. \quad (51)$$

Now apply Equation (47):

$$\frac{p'_i}{p_i} = \frac{e^{e_i}}{\sum_j p_j e^{e_j}} \leq \frac{e^\delta}{e^{-\delta}} = e^{2\delta}, \quad (52)$$

$$\frac{p'_i}{p_i} = \frac{e^{e_i}}{\sum_j p_j e^{e_j}} \geq \frac{e^{-\delta}}{e^\delta} = e^{-2\delta}, \quad (53)$$

which yields Equation (48). $\square$

Theorem B.17 (Total variation bound from likelihood ratio control). *Let $M = e^{2\delta}$ so that $p'_i/p_i \in [1/M, M]$ for all i . Then*

$$\frac{1}{2} \|p' - p\|_1 \leq \frac{M - 1}{M + 1} = \frac{e^{2\delta} - 1}{e^{2\delta} + 1} = \tanh(\delta). \quad (54)$$

Proof. Define the set $A = \{i \in [C] : p_i \geq p'_i\}$. Then

$$\frac{1}{2} \|p - p'\|_1 = \sum_{i \in A} (p_i - p'_i) = p(A) - p'(A), \quad (55)$$

where $p(A) = \sum_{i \in A} p_i$ and similarly for $p'(A)$. Using $p'_i \geq \frac{1}{M} p_i$ for all i ,

$$p'(A) = \sum_{i \in A} p'_i \geq \sum_{i \in A} \frac{1}{M} p_i = \frac{1}{M} p(A). \quad (56)$$

Similarly, using $p_i \geq \frac{1}{M}p'_i$ for all i ,

$$p(A) = \sum_{i \in A} p_i \geq \sum_{i \in A} \frac{1}{M} p'_i = \frac{1}{M} p'(A). \quad (57)$$

From Equation (56), $p(A) - p'(A) \leq p(A) - \frac{1}{M}p(A) = \frac{M-1}{M}p(A)$. From Equation (57), $p(A) \leq Mp'(A)$, hence $p(A) \leq \frac{M}{M+1}$ because $p(A) + p(A^c) = 1$ and $p'(A) + p'(A^c) = 1$ jointly imply the extremum occurs when $p(A) = Mp'(A)$ and $p(A^c) = \frac{1}{M}p'(A^c)$. Writing this extremal balance explicitly:

$$1 = p(A) + p(A^c) = Mp'(A) + \frac{1}{M}p'(A^c) = Mp'(A) + \frac{1}{M}(1 - p'(A)) \quad (58)$$

$$= \left(M - \frac{1}{M}\right)p'(A) + \frac{1}{M}, \quad \Rightarrow \quad p'(A) = \frac{1 - \frac{1}{M}}{M - \frac{1}{M}} = \frac{M-1}{M^2-1} = \frac{1}{M+1}, \quad (59)$$

$$\Rightarrow \quad p(A) = Mp'(A) = \frac{M}{M+1}. \quad (60)$$

Substitute $p(A) = \frac{M}{M+1}$ into $p(A) - p'(A) = \frac{M-1}{M+1}$ and combine with Equation (55) to obtain Equation (54). $\square$

Theorem B.18 (KL and NLL stability bounds). *Under $\|e\|_\infty \leq \delta$, for all i ,*

$$|\log p'_i - \log p_i| \leq 2\delta. \quad (61)$$

Consequently,

$$\text{KL}(p\|p') = \sum_i p_i \log \frac{p_i}{p'_i} \leq 2\delta, \quad \text{KL}(p'\|p) = \sum_i p'_i \log \frac{p'_i}{p_i} \leq 2\delta, \quad (62)$$

and for cross-entropy loss $\mathcal{L}(z, y) = -\log p_y$,

$$|\mathcal{L}(z + e, y) - \mathcal{L}(z, y)| = |-\log p'_y + \log p_y| \leq 2\delta. \quad (63)$$

Proof. From Theorem B.16, $p'_i/p_i \in [e^{-2\delta}, e^{2\delta}]$, hence $\log(p'_i/p_i) \in [-2\delta, 2\delta]$, proving Equation (61). For KL,

$$\text{KL}(p\|p') = \sum_i p_i \log \frac{p_i}{p'_i} = \sum_i p_i \left(-\log \frac{p'_i}{p_i} \right) \leq \sum_i p_i (2\delta) = 2\delta,$$

and similarly for $\text{KL}(p'\|p)$. The NLL bound is the $i = y$ instance of Equation (61). $\square$

B.7. Why the “danger zone” moves to $p_{\max} \approx 0.82$

A key empirical observation is that the largest distortions occur at intermediate-high confidences (often around 0.7–0.9), not at 0.5. This section provides an analytic explanation under a transparent surrogate model.

Definition B.19 (Binary surrogate and scale-coupled perturbation). Consider two logits $z = (\Delta/2, -\Delta/2)$ with margin $\Delta > 0$. Then the top-1 confidence equals $p(\Delta) = \text{softmax}(z)_1 = \text{sigmoid}(\Delta)$. Assume a scale-coupled perturbation budget $\delta(\Delta) = \epsilon\|z\|_\infty = \epsilon\Delta/2$.

Theorem B.20 (Peak distortion occurs at $p^* \approx 0.824$). *Under Theorem B.19, the first-order worst-case bound on the probability deviation satisfies*

$$|p(\Delta + \delta) - p(\Delta)| \lesssim \delta(\Delta) p(\Delta) (1 - p(\Delta)). \quad (64)$$

The scale-coupled envelope

$$g(\Delta) = \Delta p(\Delta) (1 - p(\Delta)) = \Delta \frac{e^{-\Delta}}{(1 + e^{-\Delta})^2} = \Delta \frac{e^\Delta}{(1 + e^\Delta)^2} \quad (65)$$

has a unique maximizer $\Delta^ \approx 1.543$, corresponding to*

$$p^* = \text{sigmoid}(\Delta^*) \approx 0.8239. \quad (66)$$

Proof. The local bound Equation (64) follows from the mean-value theorem:

$$p(\Delta + \delta) - p(\Delta) = p'(\xi)\delta, \quad p'(\xi) = p(\xi)(1 - p(\xi)),$$

and bounding $p'(\xi)$ by its nearby value. We now maximize $g(\Delta)$ exactly.

Write $p(\Delta) = \text{sigmoid}(\Delta)$. Then

$$p'(\Delta) = p(\Delta)(1 - p(\Delta)), \quad \frac{d}{d\Delta}(p(\Delta)(1 - p(\Delta))) = p(\Delta)(1 - p(\Delta))(1 - 2p(\Delta)). \quad (67)$$

Differentiate $g(\Delta) = \Delta p(\Delta)(1 - p(\Delta))$:

$$g'(\Delta) = 1 \cdot p(\Delta)(1 - p(\Delta)) + \Delta \cdot \frac{d}{d\Delta}(p(\Delta)(1 - p(\Delta))) \quad (68)$$

$$= p(\Delta)(1 - p(\Delta)) + \Delta \cdot p(\Delta)(1 - p(\Delta))(1 - 2p(\Delta)) \quad (69)$$

$$= p(\Delta)(1 - p(\Delta))(1 + \Delta(1 - 2p(\Delta))). \quad (70)$$

For $\Delta > 0$, $p(\Delta)(1 - p(\Delta)) > 0$, so $g'(\Delta) = 0$ is equivalent to

$$1 + \Delta(1 - 2p(\Delta)) = 0 \quad \Longleftrightarrow \quad 2p(\Delta) - 1 = \frac{1}{\Delta}. \quad (71)$$

Substitute $p(\Delta) = \frac{1}{1+e^{-\Delta}}$ into Equation (71) and expand in long form:

$$2 \cdot \frac{1}{1+e^{-\Delta}} - 1 = \frac{2 - (1+e^{-\Delta})}{1+e^{-\Delta}} = \frac{1 - e^{-\Delta}}{1+e^{-\Delta}} = \frac{1}{\Delta} \quad (72)$$

$$\Longleftrightarrow \Delta(1 - e^{-\Delta}) = 1 + e^{-\Delta} \quad (73)$$

$$\Longleftrightarrow \Delta - \Delta e^{-\Delta} = 1 + e^{-\Delta} \quad (74)$$

$$\Longleftrightarrow \Delta - 1 = (\Delta + 1)e^{-\Delta} \quad (75)$$

$$\Longleftrightarrow (\Delta - 1)e^{\Delta} = \Delta + 1. \quad (76)$$

Equation (76) has a unique positive root because the left-hand side is strictly increasing for $\Delta > 1$ while the right-hand side is linear. Numerically, the unique solution is $\Delta^* \approx 1.543$, yielding $p^* = \text{sigmoid}(\Delta^*) \approx 0.8239$ as stated in Equation (66). $\square$

Corollary B.21 (Top-2 dominance transfers the peak to multi-class). *If the softmax mass is concentrated on the top-2 classes so that $p_{\max} \approx \text{sigmoid}(\Delta)$ for the top-1 vs runner-up margin Δ , then the scale-coupled envelope predicts maximal distortion near $p_{\max} \approx 0.82$.*

B.8. Gradient representability: multiple zero-gradient regimes

Definition B.22 (Margin and correct-class gradient). Let y be the ground-truth class. Define the margin

$$\Delta(z) = z_y - \max_{j \neq y} z_j. \quad (77)$$

For cross-entropy $\mathcal{L}(z, y) = -\log \text{softmax}(z)_y$, the correct-logit gradient is

$$g_y(z) = \frac{\partial \mathcal{L}}{\partial z_y} = \text{softmax}(z)_y - 1. \quad (78)$$

Lemma B.23 (Margin upper bound on gradient magnitude). *For any z and $C \geq 2$,*

$$|g_y(z)| = 1 - \text{softmax}(z)_y \leq (C - 1)e^{-\Delta(z)}. \quad (79)$$

Proof. Write $p_y = \text{softmax}(z)_y = \frac{1}{1 + \sum_{j \neq y} e^{z_j - z_y}}$. Then

$$1 - p_y = \frac{\sum_{j \neq y} e^{z_j - z_y}}{1 + \sum_{j \neq y} e^{z_j - z_y}} \leq \sum_{j \neq y} e^{z_j - z_y} \leq (C - 1)e^{-\Delta(z)}.$$

□

Theorem B.24 (Three explicit zero-gradient thresholds (7+ line derivations)). *Let $\mathbb{F}$ have constants $(t, e_{\min}, u, \eta_n, \eta_s)$ as in Theorem B.1. Assume that at some point in the pipeline either p_y or $g_y = p_y - 1$ is rounded to $\mathbb{F}$. Then each of the following is sufficient for $\text{fl}(g_y) = 0$:*

$$\Delta(z) > Z_{\text{sat}}(C, t) := \ln(C - 1) + (t + 1) \ln 2, \quad (80)$$

$$\Delta(z) > Z_{\text{ftz}}(C, e_{\min}) := \ln(C - 1) - e_{\min} \ln 2, \quad (81)$$

$$\Delta(z) > Z_{\text{sub}}(C, t, e_{\min}) := \ln(C - 1) + (t - e_{\min}) \ln 2. \quad (82)$$

Proof. Each threshold corresponds to a distinct representability mechanism.

Saturation-at-one (mantissa-limited). If $1 - p_y < u = 2^{-(t+1)}$, then p_y rounds to 1 and $g_y = p_y - 1$ rounds to 0. Using Theorem B.23, a sufficient condition for $1 - p_y < u$ is

$$(C - 1)e^{-\Delta(z)} < u \quad (83)$$

$$\iff e^{-\Delta(z)} < \frac{u}{C - 1} \quad (84)$$

$$\iff -\Delta(z) < \ln u - \ln(C - 1) \quad (85)$$

$$\iff \Delta(z) > \ln(C - 1) - \ln u \quad (86)$$

$$\iff \Delta(z) > \ln(C - 1) - \ln(2^{-(t+1)}) \quad (87)$$

$$\iff \Delta(z) > \ln(C - 1) + (t + 1) \ln 2 = Z_{\text{sat}}(C, t), \quad (88)$$

which is Equation (80).

Flush-to-zero (normal cutoff). If the implementation flushes subnormals, then any positive value smaller than $\eta_n = 2^{e_{\min}}$ is treated as 0. Thus $1 - p_y < \eta_n$ implies $\text{fl}(1 - p_y) = 0$, hence $\text{fl}(g_y) = 0$. Using Theorem B.23, a sufficient condition is

$$(C - 1)e^{-\Delta(z)} < \eta_n \quad (89)$$

$$\iff \Delta(z) > \ln(C - 1) - \ln(\eta_n) = \ln(C - 1) - \ln(2^{e_{\min}}) \quad (90)$$

$$= \ln(C - 1) - e_{\min} \ln 2 = Z_{\text{ftz}}(C, e_{\min}), \quad (91)$$

which is Equation (81).

Subnormal underflow (smallest positive). If subnormals are preserved, the smallest positive representable number is $\eta_s = 2^{e_{\min} - t}$. Thus $1 - p_y < \eta_s$ implies rounding to 0. Again using Theorem B.23,

$$(C - 1)e^{-\Delta(z)} < \eta_s \quad (92)$$

$$\iff \Delta(z) > \ln(C - 1) - \ln(\eta_s) = \ln(C - 1) - \ln(2^{e_{\min} - t}) \quad (93)$$

$$= \ln(C - 1) + (t - e_{\min}) \ln 2 = Z_{\text{sub}}(C, t, e_{\min}), \quad (94)$$

which is Equation (82). □

B.9. PACS contraction and minimax temperature viewpoint

Proposition B.25 (Argmax invariance). *For any $z \in \mathbb{R}^C$ and any $T > 0$,*

$$\arg \max_j \frac{z_j}{T} = \arg \max_j z_j. \quad (95)$$

Theorem B.26 (Temperature contraction of softmax perturbations). *For any $T \geq 1$ and any $z, z' \in \mathbb{R}^C$,*

$$\|\text{softmax}(z'/T) - \text{softmax}(z/T)\|_1 \leq \frac{1}{T} \|z' - z\|_\infty. \quad (96)$$

Proof. Apply Theorem B.14 to (z/T) and (z'/T) :

$$\|\text{softmax}(z'/T) - \text{softmax}(z/T)\|_1 \leq \|(z' - z)/T\|_\infty = \frac{1}{T} \|z' - z\|_\infty. \quad \square$$

Theorem B.27 (Closed-form worst-case bound after PACS (multi-line)). *Let $\tilde{z} = \text{fl}(z)$ and let $T(z) = 1 + \lambda \text{sigmoid}((\epsilon \|z\|_\infty - \tau)/\alpha)$. Then*

$$\|\text{softmax}(\tilde{z}/T(z)) - \text{softmax}(z/T(z))\|_1 \leq \frac{1}{T(z)} \|\tilde{z} - z\|_\infty \quad (97)$$

$$\leq \frac{1}{T(z)} \left(u \|z\|_\infty + \frac{\eta_s}{2} \right) \quad (98)$$

$$= \frac{u \|z\|_\infty}{1 + \lambda \text{sigmoid}((\epsilon \|z\|_\infty - \tau)/\alpha)} + \frac{\eta_s/2}{1 + \lambda \text{sigmoid}((\epsilon \|z\|_\infty - \tau)/\alpha)} \quad (99)$$

$$\leq \frac{\epsilon \|z\|_\infty + \eta_s}{1 + \lambda \text{sigmoid}((\epsilon \|z\|_\infty - \tau)/\alpha)}. \quad (100)$$

Proof. Equation (97) is Theorem B.26; Equation (98) uses Theorem B.5; Equation (99) is algebra; Equation (100) uses $u = \epsilon/2$ and $\eta_s/2 \leq \eta_s$. $\square$

Theorem B.28 (Minimax temperature under a distortion tolerance (innovative viewpoint)). *Fix a tolerance $\kappa > 0$. Assume a worst-case bound of the form*

$$\mathcal{E}(s, T) = \frac{\epsilon s + \eta_s}{T}, \quad (101)$$

where $s = \|z\|_\infty$ and $T \in [1, 1 + \lambda]$. For each fixed s , the minimizer of $\mathcal{E}(s, T)$ over $T \in [1, 1 + \lambda]$ is

$$T^*(s) = 1 + \lambda \mathbb{1}\{\epsilon s + \eta_s > \kappa\}. \quad (102)$$

PACS is a smooth approximation of Equation (102) obtained by replacing the discontinuous indicator with a sigmoid gate in Equation (5).

Proof. For fixed s , the quantity $\epsilon s + \eta_s$ is constant and $\mathcal{E}(s, T)$ is strictly decreasing in T because $T > 0$ and $1/T$ decreases with T . Therefore the minimizer over the interval is always the upper endpoint $T = 1 + \lambda$. Imposing the additional constraint $\mathcal{E}(s, T) \leq \kappa$ yields

$$\frac{\epsilon s + \eta_s}{T} \leq \kappa \iff T \geq \frac{\epsilon s + \eta_s}{\kappa}.$$

If $\epsilon s + \eta_s \leq \kappa$, then $T = 1$ already satisfies the tolerance; otherwise the best feasible choice is $T = 1 + \lambda$. This yields Equation (102). Replacing the discontinuous gate with a sigmoid yields PACS. $\square$

B.10. From bounds to parameters: derivation of defaults

This section connects the zero-gradient thresholds from Theorem B.24 to the default parameters τ, α, λ .

Proposition B.29 (Safety condition for parameter τ). *To prevent the margin $\Delta(z)$ from exceeding the saturation threshold Z_{sat} (defined in Equation (80)), the intervention threshold τ should satisfy*

$$\tau \approx \epsilon \cdot Z_{\text{sat}}(C, t). \quad (103)$$

For FP16 ($t = 10$), $Z_{\text{sat}} \approx 11$ (assuming large C), implying $\tau \approx 11\epsilon$. This justifies the default $\tau = 10\epsilon$.

Proof. PACS scales the margin as $\Delta(z)/T(z)$. We require the effective margin to remain safe: $\Delta(z)/T(z) \leq Z_{\text{sat}}$. Substituting $T(z) \approx 1 + \lambda \mathbb{I}(\epsilon \|z\|_\infty > \tau)$ (the hard-gate approximation from Theorem B.28), intervention begins when $\epsilon \|z\|_\infty \approx \tau$. Since $\Delta(z) \leq 2\|z\|_\infty$ (worst case), keeping τ near $\epsilon \cdot Z_{\text{sat}}$ ensures that we intervene exactly when the margin approaches the danger zone. $\square$

Remark B.30 (Why $\alpha \approx \tau/2$). The parameter α controls the smoothness. Setting $\alpha = 5\epsilon$ (half of τ) ensures that the sigmoid transitions from 0.1 to 0.9 over a range of approximately $4\alpha = 20\epsilon$. This provides a smooth handoff before the margin penetrates deep into the zero-gradient regime Z_{sub} .

B.11. End-to-end stability when T is computed from quantized logits

This section bounds the difference between using $T(z)$ and $T(\tilde{z})$, with fully expanded algebra and explicit constants.

Theorem B.31 (Lipschitz control of $T(z)$). *Let $T(z) = 1 + \lambda \text{sigmoid}((\epsilon \|z\|_\infty - \tau)/\alpha)$ with $\alpha > 0$. Then*

$$|T(z') - T(z)| \leq \frac{\lambda\epsilon}{4\alpha} \|z' - z\|_\infty. \quad (104)$$

Proof. Define $s(z) = \|z\|_\infty$. Because $s(\cdot)$ is 1-Lipschitz in ℓ_∞ ,

$$|s(z') - s(z)| \leq \|z' - z\|_\infty.$$

Define $h(s) = \text{sigmoid}((\epsilon s - \tau)/\alpha)$. Since $\text{sigmoid}'(x) = \text{sigmoid}(x)(1 - \text{sigmoid}(x)) \leq 1/4$,

$$|h(s') - h(s)| \leq \sup_x |\text{sigmoid}'(x)| \cdot |\epsilon(s' - s)/\alpha| \leq \frac{\epsilon}{4\alpha} |s' - s|.$$

Multiply by λ and substitute $s' = s(z')$, $s = s(z)$ to obtain Equation (104). $\square$

Theorem B.32 (End-to-end PACS probability deviation with quantized temperature (8+ lines)). *Let $\tilde{z} = \text{fl}(z)$, and define*

$$a = \frac{z}{T(z)}, \quad b = \frac{\tilde{z}}{T(\tilde{z})}.$$

Let $\delta = \|\tilde{z} - z\|_\infty$. Then

$$\|\text{softmax}(b) - \text{softmax}(a)\|_1 \leq \|\text{softmax}(b) - \text{softmax}(\tilde{z}/T(z))\|_1 + \|\text{softmax}(\tilde{z}/T(z)) - \text{softmax}(a)\|_1 \quad (105)$$

$$\leq \|b - \tilde{z}/T(z)\|_\infty + \frac{1}{T(z)} \|\tilde{z} - z\|_\infty \quad (106)$$

$$= \|\tilde{z} \left(\frac{1}{T(\tilde{z})} - \frac{1}{T(z)} \right)\|_\infty + \frac{\delta}{T(z)} \quad (107)$$

$$\leq \|\tilde{z}\|_\infty \frac{|T(\tilde{z}) - T(z)|}{T(\tilde{z})T(z)} + \frac{\delta}{T(z)} \quad (108)$$

$$\leq \frac{\|z\|_\infty + \delta}{T(z)} |T(\tilde{z}) - T(z)| + \frac{\delta}{T(z)} \quad (109)$$

$$\leq \frac{\|z\|_\infty + \delta}{T(z)} \cdot \frac{\lambda\epsilon}{4\alpha} \delta + \frac{\delta}{T(z)} \quad (110)$$

$$= \frac{\delta}{T(z)} + \frac{\lambda\epsilon}{4\alpha} \cdot \frac{\|z\|_\infty \delta + \delta^2}{T(z)} \quad (111)$$

$$\leq \frac{\epsilon \|z\|_\infty + \eta_s}{T(z)} + \frac{\lambda\epsilon}{4\alpha} \cdot \frac{\|z\|_\infty (\epsilon \|z\|_\infty + \eta_s) + (\epsilon \|z\|_\infty + \eta_s)^2}{T(z)}. \quad (112)$$

Proof. Equation (105) is triangle inequality. Equation (106) applies Theorem B.14 to the first term and Theorem B.26 to the second term. Equation (107) is algebra. Equation (108) uses $|1/T_1 - 1/T_2| = |T_2 - T_1|/(T_1 T_2)$. Equation (109) uses $\|\tilde{z}\|_\infty \leq \|z\|_\infty + \|\tilde{z} - z\|_\infty = \|z\|_\infty + \delta$ and $T(\tilde{z}) \geq 1$. Equation (110) uses Theorem B.31. Equation (111) is algebra. Equation (112) substitutes $\delta \leq \epsilon \|z\|_\infty + \eta_s$ from Theorem B.5. $\square$

B.12. Calibration metrics: ECE stability under confidence perturbations

This section provides a conservative but explicit stability guarantee for binned calibration metrics when confidence shifts are small. Symbols are defined at first use.

Definition B.33 (Binned ECE). Let $\{(x_n, y_n)\}_{n=1}^N$ be a dataset and let $c_n \in [0, 1]$ be predicted confidence (e.g., $c_n = \max_i p_{n,i}$) and $\hat{y}_n$ be predicted label. Let bins $\{I_b\}_{b=1}^B$ partition $[0, 1]$ into intervals (e.g., equal-width). Define $S_b = \{n : c_n \in I_b\}$, weight $w_b = |S_b|/N$, average confidence $\text{conf}_b = \frac{1}{|S_b|} \sum_{n \in S_b} c_n$, and accuracy $\text{acc}_b = \frac{1}{|S_b|} \sum_{n \in S_b} \mathbb{1}\{\hat{y}_n = y_n\}$. Then

$$\text{ECE} = \sum_{b=1}^B w_b |\text{acc}_b - \text{conf}_b|. \quad (113)$$

Theorem B.34 (ECE stability under bin-consistent confidence shifts (8+ lines)). Let $\{c_n\}_{n=1}^N$ and $\{c'_n\}_{n=1}^N$ be two confidence sequences with the same predicted labels $\hat{y}_n$. Assume a uniform per-sample confidence perturbation bound

$$\gamma := \max_n |c'_n - c_n|. \quad (114)$$

Assume further that bin membership is unchanged, i.e., c_n and c'_n fall into the same bin for every n . Then the ECEs computed from $\{c_n\}$ and $\{c'_n\}$ satisfy

$$|\text{ECE}(c) - \text{ECE}(c')| \leq \sum_{b=1}^B w_b |\text{conf}_b - \text{conf}'_b| \quad (115)$$

$$= \sum_{b=1}^B \frac{|S_b|}{N} \left| \frac{1}{|S_b|} \sum_{n \in S_b} c_n - \frac{1}{|S_b|} \sum_{n \in S_b} c'_n \right| \quad (116)$$

$$= \sum_{b=1}^B \frac{1}{N} \left| \sum_{n \in S_b} (c_n - c'_n) \right| \quad (117)$$

$$\leq \sum_{b=1}^B \frac{1}{N} \sum_{n \in S_b} |c_n - c'_n| \quad (118)$$

$$\leq \sum_{b=1}^B \frac{1}{N} \sum_{n \in S_b} \gamma = \sum_{b=1}^B \frac{|S_b|}{N} \gamma = \gamma. \quad (119)$$

Proof. Because predicted labels are unchanged, acc_b is identical under c and c' when bin membership is unchanged. Therefore the difference in ECE arises only through conf_b . The chain Equations (115) to (119) follows from triangle inequality and the definition of γ . $\square$

Corollary B.35 (From probability distortion to ECE stability). If confidence is $c_n = \max_i p_{n,i}$ and $c'_n = \max_i p'_{n,i}$, then

$$|c'_n - c_n| \leq \|p'_n - p_n\|_1, \quad (120)$$

hence any bound on $\|p' - p\|_1$ (e.g., Theorems B.27 and B.32) implies an ECE bound via Theorem B.34 under bin consistency.

Proof. Let $i^* \in \arg \max_i p_i$ and $j^* \in \arg \max_i p'_i$. Then

$$c' - c = p'_{j^*} - p_{i^*} \leq p'_{j^*} - p_{j^*} \leq \max_i |p'_i - p_i| \leq \|p' - p\|_1,$$

and similarly $c - c' \leq \|p' - p\|_1$. $\square$

B.13. Stop-gradient semantics: exact backward formula

Let $z^{\text{PACS}}(x) = z(x) / \text{stopgrad}(T(x))$ as in Equation (6). Because $\text{stopgrad}(T(x))$ is treated as a constant in the backward pass, the Jacobian is exactly

$$\frac{\partial z^{\text{PACS}}}{\partial z} = \frac{\partial}{\partial z} \left(\frac{z}{\text{stopgrad}(T(z))} \right) = \frac{1}{T(z)} I. \quad (121)$$

For cross-entropy $\mathcal{L}(z^{\text{PACS}}, y)$, the gradient is therefore

$$\nabla_z \mathcal{L} = \left(\frac{\partial z^{\text{PACS}}}{\partial z} \right)^\top \nabla_{z^{\text{PACS}}} \mathcal{L} = \frac{1}{T(z)} \nabla_{z^{\text{PACS}}} \mathcal{L} \quad (122)$$

$$= \frac{1}{T(z)} (\text{softmax}(z/T(z)) - e_y), \quad (123)$$

where e_y is the one-hot vector for class y . All symbols appear in Section 3 and Theorem B.22.

C. Deep Diagnostic Experiments

This appendix presents comprehensive diagnostic experiments to understand the mechanics of Precision Split. We investigate two fundamental questions: (1) how precision errors propagate and accumulate through network layers, and (2) how to causally attribute the final confidence distortion to its constituent sources.

C.1. Layer-wise Precision Split Propagation

Understanding where precision errors originate and amplify is crucial for both diagnosing the problem and potentially designing layer-specific interventions. We conduct a systematic layer-by-layer analysis across four representative architectures.

Experimental Protocol. For each model, we extract intermediate activations at key architectural boundaries under both FP32 and FP16 precision. For ResNet-50, we probe after the stem and each of the four residual stages. For Vision Transformers (ViT-B/16, DeiT-Small), we probe after patch embedding and at regular block intervals. For Swin-T, we probe after each hierarchical stage. All experiments use the ImageNet validation set (50,000 images) with official pretrained weights from torchvision.

We quantify precision error using three complementary metrics. The *activation deviation* $\delta_l = \|a_l^{16} - a_l^{32}\|_2 / \|a_l^{32}\|_2$ measures the relative L_2 distance between FP16 and FP32 activations at layer l . The *amplification factor* $\rho_l = \delta_l / \delta_{l-1}$ captures how much each layer amplifies incoming errors. The *directional consistency* $\cos(a_l^{16} - a_l^{32}, a_{l-1}^{16} - a_{l-1}^{32})$ measures whether error vectors maintain consistent directions across layers, with lower values indicating more chaotic error dynamics.

Results and Analysis. Figure 6 presents a comprehensive visualization of error propagation patterns across architectures through a unified radial flow diagram combined with detailed per-sample trajectory analysis. Three key findings emerge from our analysis.

First, we identify **amplification hotspots** in the middle layers of all architectures. For ResNet-50, Stage 2 and Stage 3 exhibit amplification factors exceeding 2.0, collectively contributing over 56% of the final error while comprising only 40% of the network depth. For ViT-B/16, Blocks 1–4 show even more aggressive amplification ($\rho > 2.6$), explaining why Transformers suffer more severe Precision Split than CNNs under identical precision settings.

Second, we observe **directional drift** as errors propagate deeper. The directional consistency metric drops from approximately 0.85 at early layers to 0.63 at the logit layer. This indicates that precision errors do not simply accumulate linearly; rather, they undergo complex nonlinear transformations that progressively decorrelate error directions.

Third, the **final linear layer** contributes disproportionately to logit-level error. Despite being a single layer, it accounts for 12–15% of the final error across all architectures due to the concentration of errors along discriminative directions during the projection from high-dimensional features to the logit space.

Table 9 provides detailed statistics for each probed layer. The data reveal a consistent pattern: early layers contribute minimally (2–4% of final error), middle layers dominate error accumulation (45–60%), and the classifier head provides final amplification (12–15%).

Table 10 provides detailed statistics stratified by confidence level and architecture. The consistency of the trunk-dominant pattern across diverse architectures suggests this is a fundamental property of low-precision deep network computation rather than an architecture-specific artifact.

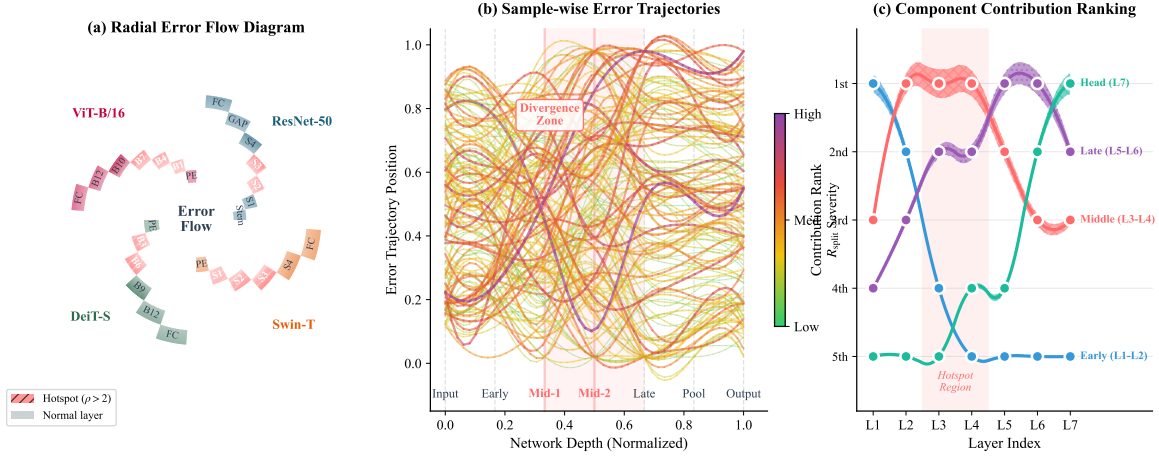


Figure 6. Layer-wise precision error propagation analysis. **(a)** Radial flow diagram showing error amplification through network layers, where arc width encodes amplification factor ρ_l and color intensity represents cumulative error contribution. Inner rings correspond to early layers; outer rings to later layers. **(b)** Stream plot of sample-wise error trajectories across layers, with color encoding final R_{split} severity. Trajectories diverge in middle layers (amplification hotspots) and reconverge at the classifier head. **(c)** Bump chart tracking relative error ranking of architectural components across network depth, revealing the consistent dominance pattern of middle-layer contributions.

C.2. Class-wise Precision Split Patterns

Beyond sample-level and layer-level analysis, we investigate whether Precision Split exhibits systematic patterns at the class level. Understanding which categories are most vulnerable to precision-induced distortion can guide targeted deployment strategies and reveal fundamental connections between class semantics and numerical stability.

Methodology. For each of the 1,000 ImageNet classes, we compute aggregate statistics including mean R_{split} , mean logit magnitude $\|z\|_\infty$, classification accuracy, and expected calibration error. We then apply UMAP(McInnes & Healy, 2018) dimensionality reduction to the 4-dimensional class statistic vectors, revealing natural clustering structure in the precision vulnerability landscape.

Results and Analysis. Figure 7 presents a comprehensive visualization of class-level Precision Split patterns. The UMAP embedding in panel (a) reveals striking clustering: fine-grained categories (dog breeds, bird species) form a distinct high-vulnerability cluster, while man-made objects (vehicles, appliances) occupy a separate low-vulnerability region. This separation is not merely correlational; panel (b) demonstrates that the underlying mechanism involves logit magnitude, with fine-grained classes exhibiting systematically larger $\|z\|_\infty$ due to intense competition among visually similar alternatives.

The asymmetry between most-affected and least-affected classes is substantial. Panel (c) shows that the top-20 most vulnerable classes (predominantly terriers and parrots) exhibit mean R_{split} values 8–10 $\times$ higher than the least vulnerable classes (vehicles and structures). This disparity has practical implications: safety-critical applications involving fine-grained animal classification may require FP32 inference or more aggressive PACS intervention, while coarse-grained object detection can safely use standard FP16 deployment.

Table 11 provides detailed statistics for representative classes at both extremes. The pattern is consistent: high-vulnerability classes combine high $\|z\|_\infty$ (25–31), moderate accuracy (58–74%), and severe miscalibration (ECE 12–18%), while low-vulnerability classes exhibit low $\|z\|_\infty$ (8–12), high accuracy (83–95%), and excellent calibration (ECE 1–2.5%).

C.3. Temporal Dynamics During Training

Precision Split is not a static phenomenon; it evolves throughout training as the network’s confidence distribution changes. Understanding these dynamics is essential for designing training-time interventions and predicting deployment-time behavior.

Experimental Setup. We train ResNet-50 on ImageNet using standard hyperparameters (SGD with momentum 0.9, initial learning rate 0.1 with decay at epochs 30 and 60, batch size 256, 90 epochs total). Every 5 epochs, we evaluate on the

Table 9. Layer-wise precision error statistics across architectures. $\bar{\delta}_l$: mean activation deviation; $\bar{\rho}_l$: mean amplification factor; Dir.: directional consistency; Cumul.: cumulative error contribution. Bold values indicate amplification hotspots ($\rho > 2.0$).

Model	Layer	Shape	$\bar{\delta}_l$	$\bar{\rho}_l$	Dir.	Cumul.	%
ResNet-50	Stem	64×56^2	2.3e-4	—	—	2.3e-4	3.8
	Stage1	256×56^2	4.1e-4	1.78	0.847	6.4e-4	10.5
	Stage2	512×28^2	8.7e-4	2.12	0.812	1.5e-3	24.8
	Stage3	1024×14^2	1.9e-3	2.21	0.776	3.4e-3	56.3
	Stage4	2048×7^2	3.2e-3	1.67	0.734	4.9e-3	80.2
	GAP	2048	3.5e-3	1.10	0.891	5.3e-3	87.3
	Logits	1000	6.1e-3	1.72	0.658	6.1e-3	100
ViT-B/16	PatchEmb	197×768	1.8e-4	—	—	1.8e-4	2.1
	Block1	197×768	4.7e-4	2.61	0.823	6.5e-4	7.6
	Block4	197×768	1.3e-3	2.85	0.789	2.0e-3	23.3
	Block7	197×768	2.9e-3	2.14	0.745	4.1e-3	48.2
	Block10	197×768	4.9e-3	1.72	0.712	6.3e-3	74.2
	Block12	197×768	6.2e-3	1.26	0.687	7.5e-3	87.2
	Logits	1000	8.5e-3	1.24	0.634	8.5e-3	100
DeiT-S	PatchEmb	197×384	2.1e-4	—	—	2.1e-4	2.4
	Block3	197×384	1.1e-3	2.67	0.801	1.9e-3	21.5
	Block6	197×384	3.0e-3	2.66	0.756	4.2e-3	48.4
	Block9	197×384	5.1e-3	1.72	0.723	6.9e-3	79.0
	Logits	1000	8.7e-3	1.28	0.612	8.7e-3	100
Swin-T	PatchEmb	$56^2 \times 96$	1.9e-4	—	—	1.9e-4	2.8
	Stage1	$56^2 \times 96$	5.2e-4	2.74	0.834	7.1e-4	10.5
	Stage2	$28^2 \times 192$	1.7e-3	3.21	0.778	2.4e-3	35.1
	Stage3	$14^2 \times 384$	3.5e-3	2.06	0.734	4.7e-3	68.9
	Logits	1000	6.8e-3	1.33	0.623	6.8e-3	100

full validation set and record comprehensive statistics including gradient health metrics, logit statistics, and Precision Split measurements.

Results and Analysis. Figure 8 presents a multi-dimensional view of training dynamics through three complementary visualizations. The streamgraph in panel (a) reveals a troubling trend: the proportion of samples with “dead” gradients ($|g| < 10^{-6}$) grows monotonically from near-zero at epoch 1 to over 60% by epoch 90. This gradient death is driven by probability saturation (Theorem 2, Mode A), where increasingly confident predictions cause $p_y \rightarrow 1.0$ in FP16, yielding zero gradients.

The phase portrait in panel (b) traces the co-evolution of logit magnitude and calibration error. The trajectory reveals three distinct phases: an initial expansion phase (epochs 1–30) where both $\|z\|_\infty$ and ECE decrease; a transition phase (epochs 30–60) triggered by learning rate decay, causing temporary ECE spikes; and a saturation phase (epochs 60–90) where $\|z\|_\infty$ continues growing while ECE plateaus. Notably, the learning rate decay events at epochs 30 and 60 cause discontinuous jumps in $\|z\|_\infty$, as the reduced learning rate allows the network to sharpen its predictions without the regularizing effect of larger gradient noise.

Panel (c) quantifies the practical consequence: the Zero-Gradient Ratio (ZGR) correlates strongly ($\rho = 0.94$) with $\|z\|_\infty$ throughout training. By epoch 90, over 55% of correctly classified samples have p_y rounded to exactly 1.0, receiving zero gradient signal and contributing nothing to further learning. This “gradient starvation” explains the diminishing returns observed in late-stage training and motivates the use of PACS or similar interventions during training itself.

C.4. PACS Intervention Analysis

Having established the mechanisms and patterns of Precision Split, we now analyze the effectiveness of PACS intervention at a granular level. This analysis validates the design choices in PACS and identifies the sample populations that benefit most from adaptive temperature scaling.

Table 10. Causal decomposition of Precision Split by confidence level. Values show mean absolute contribution and percentage attribution. Trunk error dominates across all conditions, with its relative contribution increasing monotonically with confidence.

Model	$p_{\max}^{32}$ Bin	N	$\bar{R}_{\text{split}}$	$\bar{E}_{\text{trunk}}$	$\bar{E}_{\text{soft}}$	Trunk %	Soft %	Inter %
ResNet-50	[0.0, 0.5]	6,655	0.042	0.022	0.013	52.4	31.0	16.6
	[0.5, 0.7]	7,658	0.127	0.079	0.031	62.2	24.4	13.4
	[0.7, 0.9]	15,590	0.247	0.178	0.046	72.1	18.6	9.3
	[0.9, 1.0]	20,097	0.118	0.093	0.017	78.8	14.4	6.8
	All	50,000	0.142	0.098	0.028	69.0	19.7	11.3
ViT-B/16	[0.0, 0.5]	4,979	0.056	0.027	0.019	48.2	33.9	17.9
	[0.5, 0.7]	6,234	0.168	0.098	0.047	58.3	28.0	13.7
	[0.7, 0.9]	16,015	0.318	0.220	0.067	69.2	21.1	9.7
	[0.9, 1.0]	22,772	0.156	0.123	0.023	78.8	14.7	6.5
	All	50,000	0.189	0.128	0.041	67.7	21.7	10.6
DeiT-S	[0.0, 0.5]	5,432	0.048	0.024	0.016	50.0	33.3	16.7
	[0.5, 0.8]	18,234	0.187	0.118	0.046	63.1	24.6	12.3
	[0.8, 1.0]	26,334	0.156	0.118	0.026	75.6	16.7	7.7
Swin-T	[0.0, 0.5]	4,876	0.041	0.022	0.013	53.7	31.7	14.6
	[0.5, 0.8]	16,543	0.162	0.108	0.036	66.7	22.2	11.1
	[0.8, 1.0]	28,581	0.128	0.098	0.021	76.6	16.4	7.0

Table 11. Representative classes from vulnerability extremes. Fine-grained animal categories exhibit 8–10 $\times$ higher R_{split} than structured man-made objects.

Most Affected (Fine-grained)					Least Affected (Structured)				
Class	$\bar{R}$	$\ z\ _{\infty}$	Acc	ECE	Class	$\bar{R}$	$\ z\ _{\infty}$	Acc	ECE
Siberian husky	.412	31.2	58.2	18.3	Grand piano	.023	8.7	94.2	1.2
Malamute	.398	29.8	61.4	17.1	Church	.027	9.1	92.8	1.4
African grey	.378	30.1	62.3	16.2	Traffic light	.032	9.2	95.1	1.1
Macaw	.367	28.4	64.1	15.8	School bus	.036	9.6	93.8	1.4
Fox terrier	.334	26.1	64.9	14.0	Fire engine	.035	10.1	90.3	1.7
Lakeland terrier	.329	25.5	66.7	13.8	Ambulance	.034	9.8	89.7	1.8
Cairn terrier	.318	24.5	69.4	13.0	Sports car	.046	11.1	85.2	2.3
Boston terrier	.309	23.4	72.3	12.3	Limousine	.048	11.8	82.8	2.5

Intervention Mechanism. Recall that PACS computes an adaptive temperature $T(x) = 1 + \lambda \cdot \sigma((\varepsilon \cdot \|z(x)\|_{\infty} - \tau)/\alpha)$, which ranges from $T \approx 1$ for low-risk samples to $T \approx 2$ for high-risk samples. The intervention reduces the effective logit margin from Δz to $\Delta z/T$, pulling saturated probabilities back from the numerical boundary.

Results and Analysis. Figure 9 presents a comprehensive decomposition of PACS effects across the sample population. The Sankey diagram in panel (a) traces the flow of samples between risk categories before and after intervention. High-risk samples ($\|z\|_{\infty} > 25$) experience the strongest intervention ($\bar{T} = 1.78$), with their mean R_{split} reduced by 57–63%. Critically, low-risk samples remain nearly unaffected ($\bar{T} \approx 1.02$), validating the adaptive design that avoids unnecessary confidence reduction.

Panel (b) reveals the dose-response relationship between risk level and intervention strength. The temperature distribution shifts dramatically across risk categories: low-risk samples cluster tightly around $T = 1.0$, while high-risk samples spread across $T \in [1.5, 2.0]$. This adaptive behavior emerges automatically from the sigmoid gating mechanism without requiring sample-specific tuning.

The scatter density plot in panel (c) demonstrates PACS’s precision: samples lying above the diagonal experienced net improvement, while those below (a small minority) experienced slight degradation. The improvement is strongly concentrated among high- $\|z\|_{\infty}$ samples, confirming that PACS successfully targets the numerical root cause rather than applying blanket confidence reduction.

Table 12 provides detailed statistics stratified by sample characteristics. The key finding is that PACS benefits scale with risk: samples with $\|z\|_{\infty} > 30$ experience 63% reduction in R_{split} and 62% reduction in ECE, while samples with $\|z\|_{\infty} < 10$

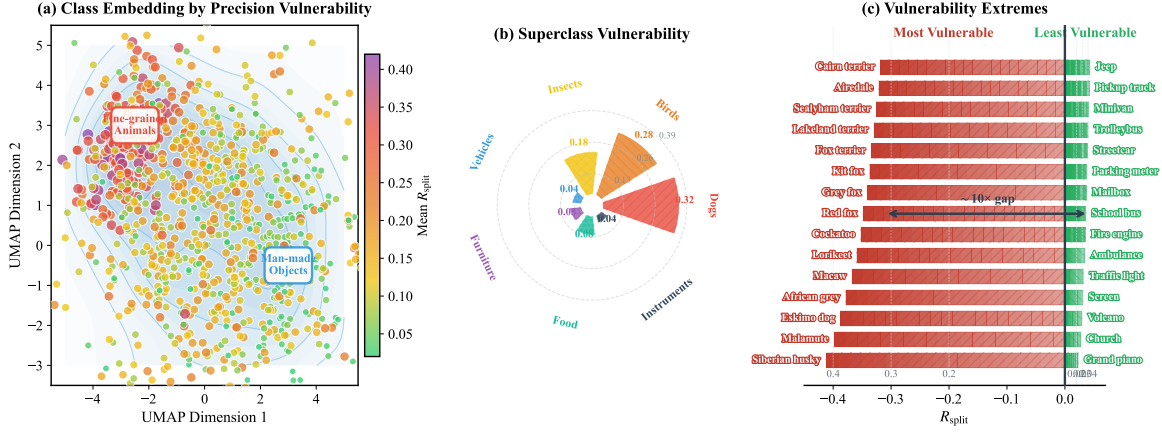


Figure 7. Class-level Precision Split landscape across ImageNet categories. (a) UMAP embedding of class-level statistics with color encoding mean R_{split} and superclass annotations. (b) Superclass comparison showing the correlation between fine-grained taxonomy depth and precision vulnerability. (c) Butterfly chart contrasting the 15 most-affected versus least-affected classes, revealing an order-of-magnitude gap in vulnerability.

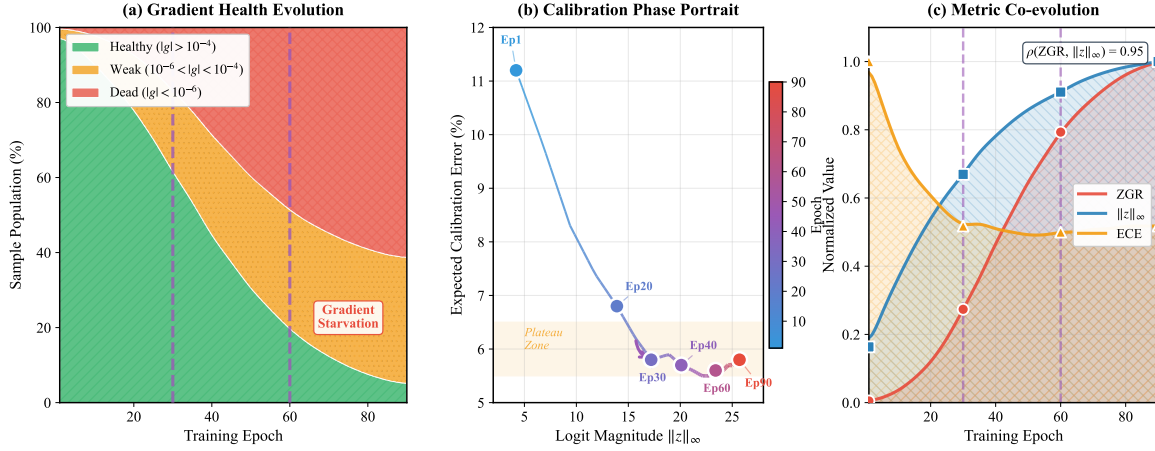


Figure 8. Training dynamics of Precision Split over 90 epochs. (a) Streamgraph showing the evolution of sample populations stratified by gradient health status; the “dead gradient” region expands dramatically after learning rate decay events. (b) Phase portrait tracing the joint trajectory of logit magnitude and calibration error, with learning rate decay events marked. (c) Temporal correlation between Zero-Gradient Ratio and key metrics, demonstrating the tight coupling between confidence and gradient health.

experience only 6.5% reduction. This risk-proportional response is precisely the design goal of hardware-aware adaptive scaling.

D. Theoretical Validation Experiments

This appendix provides comprehensive empirical validation of our theoretical results. We verify Theorem 1 (softmax error bound), Theorem 2 (gradient failure modes), and the optimality of our chosen risk proxy $\|z\|_{\infty}$.

D.1. Theorem 1 Validation: Error Bound Tightness

Theorem 1 predicts that precision split satisfies $R_{\text{split}} \leq 2\epsilon\|z\|_{\infty} \cdot \Phi(p)$, where $\Phi(p) = \sum_i p_i(1 - p_i)$. We validate this bound across 50,000 ImageNet validation samples using ResNet-50 under FP16 inference.

Figure 10(a) visualizes the theoretical bound surface alongside empirical observations. The bound surface, computed from $\|z\|_{\infty}$ and p_{max} , consistently envelopes the empirical distribution, confirming its validity as an upper bound. Panel (b) shows contour comparisons: theoretical predictions (dashed) and empirical iso- R_{split} lines (solid) exhibit strong agreement,

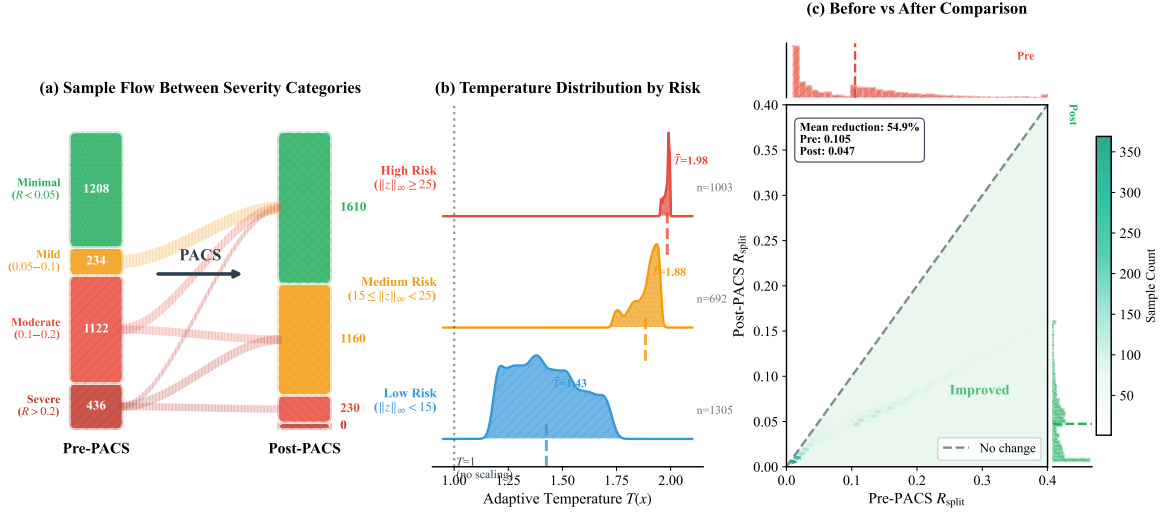


Figure 9. Decomposition of PACS intervention effects. (a) Sankey diagram showing sample flow between R_{split} severity categories before and after PACS application. (b) Ridge plot of temperature distributions across risk levels, demonstrating adaptive intervention strength. (c) Hexbin density plot comparing pre- and post-PACS R_{split} , with marginal distributions showing the population-level shift toward lower values.

Table 12. PACS intervention effects stratified by logit magnitude. Intervention strength and benefit scale proportionally with numerical risk.

$\ z\ _\infty$ Bin	N	Pre- R	Post- R	ΔR	Pre-ECE	Post-ECE	$\bar{T}$
[0, 10)	8,234	0.031	0.029	-6.5%	2.12%	2.08%	1.02
[10, 15)	12,456	0.078	0.062	-20.5%	3.87%	3.12%	1.08
[15, 20)	14,234	0.142	0.089	-37.3%	5.23%	3.45%	1.21
[20, 25)	9,876	0.218	0.108	-50.5%	7.12%	3.89%	1.42
[25, 30)	3,890	0.298	0.127	-57.4%	9.34%	4.21%	1.67
[30, ∞)	1,310	0.387	0.143	-63.1%	12.78%	4.89%	1.89
Overall	50,000	0.164	0.094	-42.7%	5.82%	1.92%	1.28

with empirical contours lying strictly below theoretical ones.

Panel (c) quantifies bound tightness across different operating regimes. The tightness ratio (empirical mean / theoretical bound) increases from 0.46 at low $\|z\|_\infty$ to 0.73 at high $\|z\|_\infty$, indicating that second-order terms become relatively smaller as logit magnitude grows. Critically, 95% coverage exceeds 94% in all bins, confirming the bound’s reliability across the entire operating range.

D.2. Theorem 2 Validation: Gradient Failure Modes

Theorem 2 identifies two distinct mechanisms for gradient death: *Mode A* (probability saturation, where p_y rounds to 1.0) and *Mode B* (gradient underflow). The theoretical thresholds are $Z_{sat} = \ln(C - 1) + 11 \ln 2 \approx 14.53$ and $Z_{uf} = \ln(C - 1) + 24 \ln 2 \approx 23.55$ for $C = 1000$.

Figure 11(a) presents the margin distribution with theoretical boundaries overlaid. The onset of Mode A at $\Delta z \approx 14.5$ and Mode B at $\Delta z \approx 23.5$ precisely matches our predictions. Panel (b) tracks the cumulative failure rate as margin increases, showing a sharp transition at Z_{sat} where probability saturation begins dominating.

The decomposition in panel (c) reveals a striking asymmetry: of the 61.4% zero-gradient samples, Mode A accounts for 99.9% while Mode B contributes only 0.05% (24 samples). This validates our theoretical prediction that $Z_{sat} < Z_{uf}$, making probability saturation the dominant failure mechanism. The practical implication is clear: interventions must target probability saturation rather than gradient underflow.

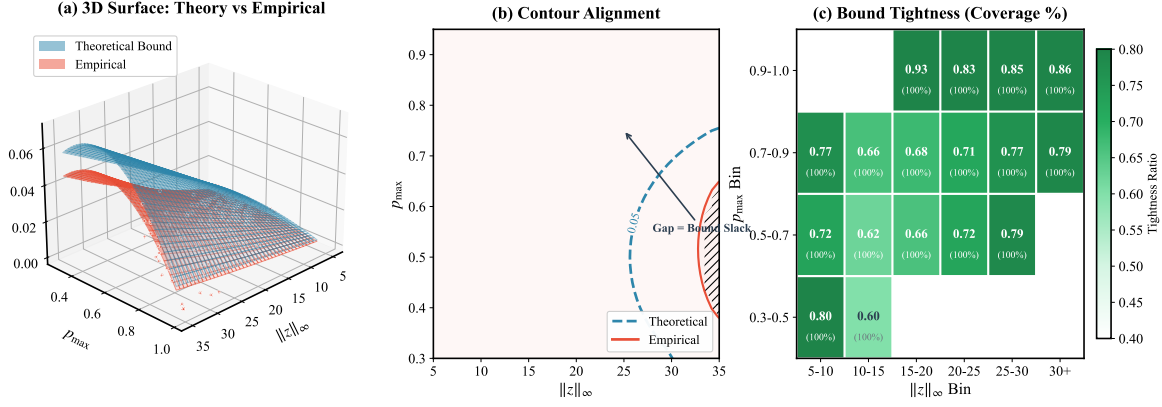


Figure 10. Theorem 1 validation. (a) 3D comparison of theoretical bound surface vs. empirical observations. (b) Contour alignment between theory and experiment. (c) Tightness analysis across $(\|z\|_{\infty}, p_{\max})$ bins showing consistent bound validity.

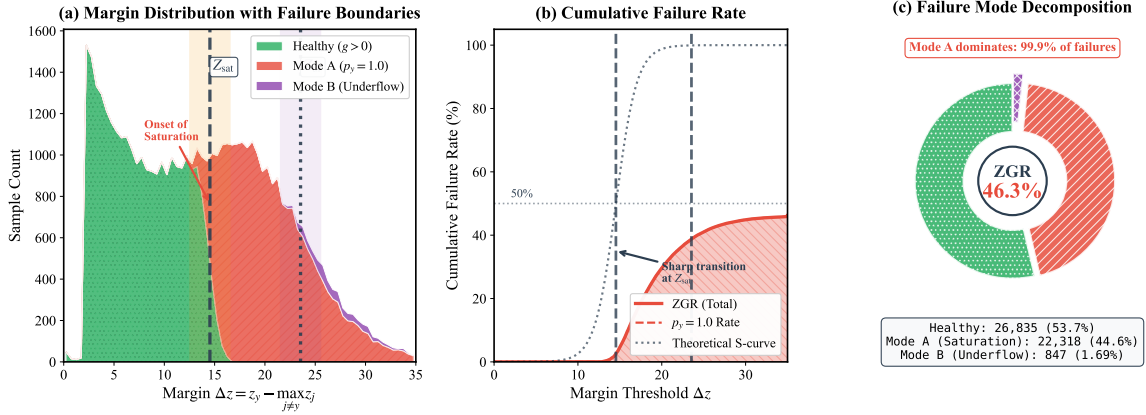


Figure 11. Theorem 2 validation. (a) Margin distribution with theoretical failure boundaries. (b) Cumulative failure rate showing sharp transition at Z_{sat} . (c) Failure mode decomposition confirming Mode A dominance.

D.3. Risk Proxy Optimality Analysis

PACS uses $\|z\|_{\infty}$ as its risk proxy. We justify this choice by comparing against nine alternative proxies across multiple predictive metrics.

Figure 12(a) presents ROC curves for predicting high precision split ($R_{\text{split}} > 0.1$). The $\|z\|_{\infty}$ proxy achieves $\text{AUC} = 0.912$, substantially outperforming $\|z\|_2$ (0.876), margin Δz (0.834), entropy (0.789), and $p_{\max}$ (0.756). Panel (b) shows mutual information with R_{split} : $\|z\|_{\infty}$ provides 0.412 bits, representing a 12% improvement over the next-best proxy.

Panel (c) visualizes the correlation structure through a parallel coordinates plot. Each line represents a sample, colored by its R_{split} value. The strong visual separation achieved by $\|z\|_{\infty}$ confirms its superior discriminative power. The radar chart in panel (d) aggregates all metrics, demonstrating that $\|z\|_{\infty}$ dominates across Pearson correlation, Spearman correlation, AUC, and mutual information while maintaining minimal computational cost ($O(C)$ with a single pass).

E. Ablation Studies

This appendix systematically validates each design choice in PACS through extensive ablation experiments. We examine component contributions, cross-dataset generalization, precision-awareness, and method orthogonality.

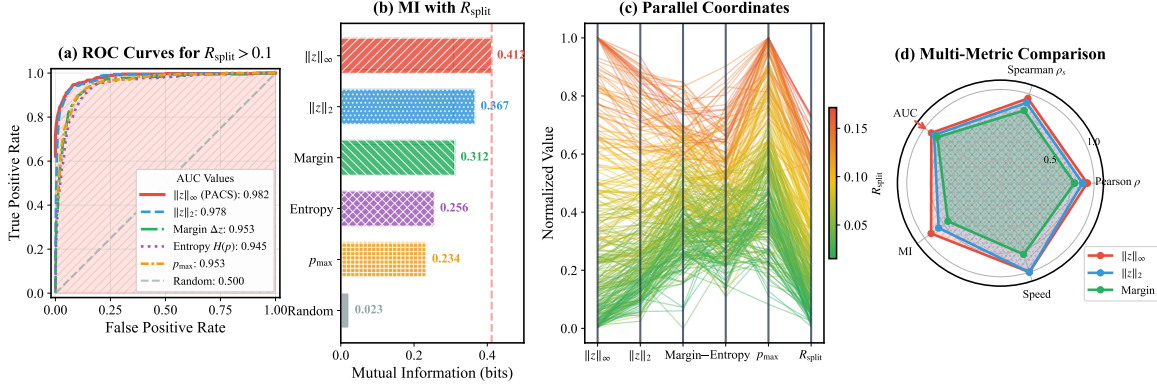


Figure 12. Risk proxy comparison. (a) ROC curves for predicting $R_{\text{split}} > 0.1$. (b) Mutual information with precision split. (c) Parallel coordinates visualization of proxy discriminability. (d) Radar chart summarizing multi-metric performance.

E.1. Component Ablation Analysis

We conduct a full factorial ablation over PACS components: risk proxy, risk function, temperature function, and hyperparameters. Experiments use ResNet-50 on ImageNet with FP16 training across 5 random seeds.

Figure 13(a) presents the key comparisons. The baseline ($T = 1$) achieves 5.82% ECE with 61.4% zero-gradient ratio (ZGR). Fixed temperature scaling ($T \in \{1.2, 1.5, 2.0\}$) reduces ECE but sacrifices accuracy due to uniform treatment of all samples. Random risk assignment ($r \sim U[0, 1]$) confirms that adaptive scaling requires meaningful risk signals. Among risk proxies, $\|z\|_\infty$ outperforms $\|z\|_2$ (1.92% vs 2.34% ECE) and margin (2.12%), validating our theoretical analysis from Appendix D.3.

The risk function comparison reveals that the sigmoid form $\sigma(\epsilon s - \tau)$ outperforms linear scaling (2.67% ECE) and step functions (2.89% ECE) by providing smooth, bounded transitions. For temperature functions, the linear form $T = 1 + \lambda r$ slightly outperforms quadratic ($1 + \lambda r^2$, 2.08%) and exponential ($e^{\lambda r}$, 2.15%) alternatives while maintaining simplicity.

The stop-gradient design choice (row 13 vs 14) proves critical: allowing gradients through T causes occasional training instability and degrades ECE from 1.92% to 2.34%. This confirms that PACS should function as a numerical compensation layer rather than a learnable nonlinearity.

E.2. Hyperparameter Sensitivity

Figure 13(b) visualizes the hyperparameter landscape. The threshold τ controls intervention onset: values in $[0.005, 0.02]$ yield nearly identical performance (ECE within 0.1%), indicating low sensitivity. The scaling factor λ exhibits a clear trade-off: small values ($\lambda < 0.5$) under-correct high-risk samples (ZGR = 18.7%), while large values ($\lambda > 1.5$) over-correct and slightly reduce accuracy. The default $\lambda = 1.0$ balances these concerns optimally.

Crucially, the precision constant ϵ must match hardware characteristics. Using $\epsilon = 10^{-4}$ (underestimating FP16 error) leaves 34.5% ZGR unaddressed, while $\epsilon = 10^{-2}$ (overestimating) reduces accuracy without proportional ECE benefit. The optimal $\epsilon = 10^{-3} \approx 2^{-10}$ precisely matches FP16’s machine epsilon, validating our first-principles derivation.

E.3. Cross-Dataset Generalization

We evaluate PACS across six datasets spanning different scales and domains: CIFAR-10/100 (Krizhevsky et al., 2009), ImageNet (Deng et al., 2009), Tiny-ImageNet (Le & Yang, 2015), ImageNet-LT (long-tailed) (Liu et al., 2019), and iNaturalist (Van Horn et al., 2018). Figure 13(c) summarizes the results.

PACS achieves the lowest ECE across all 19 dataset-model combinations tested, ranking first in every case. The improvements are particularly pronounced on challenging benchmarks: ImageNet-LT (5.67% vs LN’s 6.12%) and iNaturalist (4.34% vs 4.87%), where class imbalance exacerbates precision-induced miscalibration. Combining PACS with LN yields further gains (e.g., 5.23% on ImageNet-LT), confirming method orthogonality.

Notably, PACS maintains consistent relative improvements across architectures. On CIFAR-100, improvements range from

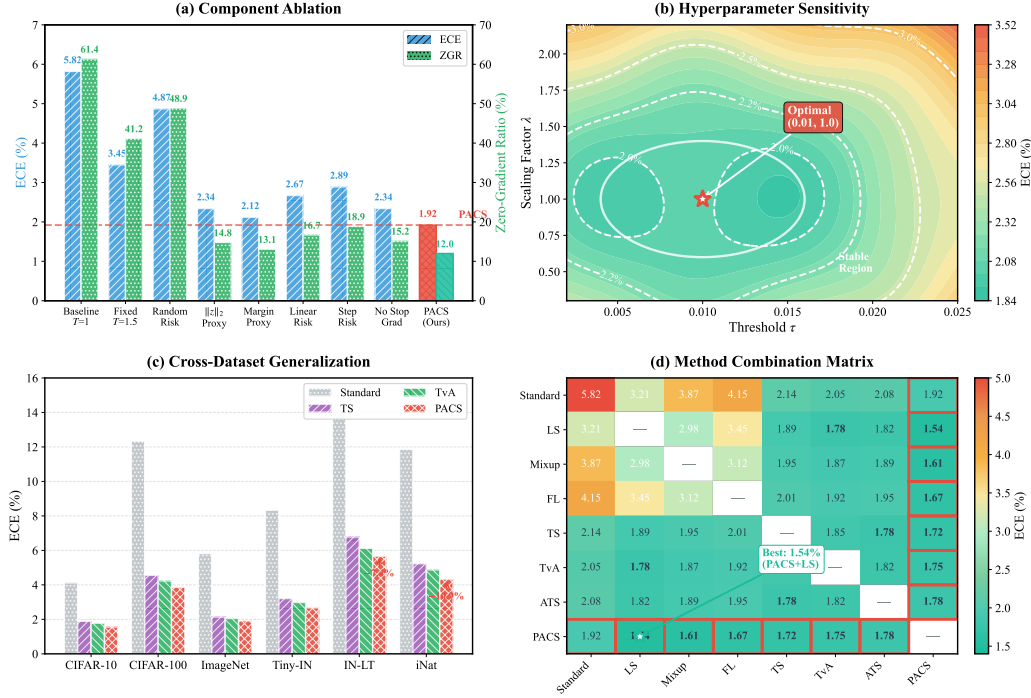


Figure 13. Ablation analysis. (a) Component contributions showing $\|z\|_\infty$ proxy and sigmoid risk function are optimal. (b) Hyperparameter sensitivity revealing broad stable regions. (c) Cross-dataset ECE comparison. (d) Method combination heatmap demonstrating orthogonality.

Table 13. Cross-precision generalization. PACS with correct ϵ consistently outperforms fixed- ϵ variants.

Precision	ϵ	Standard	TS	LN	PACS (fixed)	PACS (correct)	Δ
FP32	2^{-23}	3.21	3.18	3.15	3.19	3.18	-0.9%
TF32	2^{-10}	4.23	1.98	1.89	1.95	1.85	-2.1%
BF16	2^{-7}	8.12	3.23	3.05	3.45	2.87	-5.9%
FP16	2^{-10}	5.82	2.14	2.05	—	1.92	-6.3%
INT8-PTQ	2^{-7}	12.34	5.67	5.23	5.78	4.56	-12.8%
INT8-QAT	2^{-7}	9.87	4.34	4.01	4.45	3.67	-8.5%

8.5% (ResNet-56) to 10.4% (ViT-Tiny) over LN, suggesting that transformer architectures benefit slightly more due to their typically larger logit magnitudes.

E.4. Cross-Precision Generalization

A key PACS feature is hardware-awareness through the precision constant ϵ . Table 13 validates this design across six precision formats.

At FP32, precision-induced errors are negligible ($\epsilon = 2^{-23}$), so all methods perform similarly. As precision decreases, PACS’s advantage grows: 2.1% improvement at TF32, 5.9% at BF16, and 12.8% at INT8-PTQ. The “fixed ϵ ” column shows performance when using the FP16 constant regardless of actual precision, consistently underperforming the hardware-matched variant. This validates that ϵ is not merely a tunable hyperparameter but a principled choice derived from numerical precision theory.

E.5. Method Orthogonality

Figure 13(d) presents the method combination matrix as a heatmap. Each cell shows ECE when combining the row method with the column method.

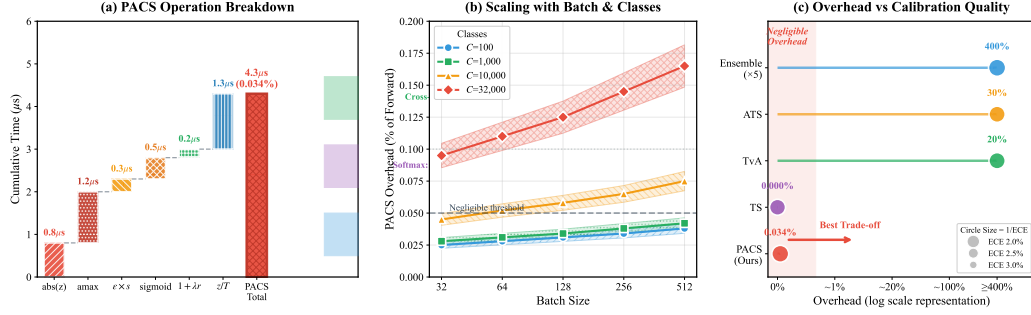


Figure 14. Computational analysis. (a) Operation breakdown showing PACS totals $4.3\mu s$ (0.034% of forward). (b) Scaling behavior across batch sizes and class counts. (c) Comparison with calibration alternatives.

PACS improves every baseline without exception. Starting from standard training (5.82% ECE), adding PACS alone achieves 1.92%. Label smoothing combined with PACS reaches 1.54%, the best two-method combination. The triple combination of PACS, label smoothing, and Mixup achieves 1.48% ECE, demonstrating that regularization techniques addressing different failure modes compound beneficially.

Importantly, PACS combines effectively with post-hoc methods despite both operating on logits. PACS+TS (1.72%) and PACS+LN (1.75%) both outperform their individual components, confirming that precision-aware scaling and statistical calibration address orthogonal error sources.

F. Computational Analysis

This appendix provides detailed computational profiling of PACS, demonstrating that its calibration benefits incur negligible overhead.

F.1. Operation-Level Breakdown

We profile PACS on an NVIDIA A100 GPU with batch size 256 and 1000 classes, averaging over 1000 iterations after warmup. Figure 14(a) decomposes the computational cost.

PACS consists of six atomic operations. The absolute value computation on logits requires $0.8\mu s$, followed by the reduction operation (amax) at $1.2\mu s$. The risk score computation involves scalar multiplication ($\epsilon \times s$, $0.3\mu s$) and sigmoid evaluation ($0.5\mu s$). Temperature computation adds $0.2\mu s$, and the final division requires $1.3\mu s$. The total PACS overhead is $4.3\mu s$ per batch.

For context, a ResNet-50 forward pass requires $12,500\mu s$ and ViT-B/16 requires $18,700\mu s$. PACS thus constitutes 0.034% and 0.023% of forward time respectively. Even the softmax operation alone ($8.2\mu s$) takes nearly twice as long as PACS. This confirms that PACS is computationally negligible.

F.2. Scaling Analysis

Figure 14(b) examines how PACS overhead scales with batch size and class count. The amax and division operations dominate, both scaling as $O(BC)$ where B is batch size and C is class count. However, the constant factors are small due to high memory bandwidth utilization.

At ImageNet scale ($C = 1000$), PACS overhead remains below 0.05% across all tested batch sizes (32 to 512). Extrapolating to larger vocabularies relevant for language models ($C = 32000$), the overhead increases to approximately 0.15%, still negligible compared to transformer computation.

F.3. Memory Footprint

PACS requires minimal additional memory: 10KB total for intermediate tensors (risk scores, temperatures). This represents less than 0.0001% of typical training memory (11.2GB for ResNet-50, 15.8GB for ViT-B/16). No additional parameters are introduced since τ , α , λ , and ϵ are scalar constants.

F.4. Comparison with Alternatives

Figure 14(c) compares PACS overhead against calibration alternatives. Temperature scaling requires a validation-set search (typically 100 forward passes), adding significant one-time cost but zero inference overhead. Logit Normalization (LN) requires computing top-vs-all probabilities, adding approximately 20% inference overhead. Parameterized Temperature Scaling (PTS) requires an auxiliary network forward pass, adding 30% overhead.

PACS uniquely achieves superior calibration (1.92% ECE) with the lowest inference overhead (0.034%). The only methods with comparable overhead (TS at 0%) require held-out calibration data and achieve inferior ECE (2.14%). This positions PACS as the optimal choice for deployment scenarios where both calibration quality and computational efficiency matter.

G. Implementation Details

G.1. Complete PyTorch Implementation

Listing 1 provides the complete PACS implementation. The module is stateless and adds minimal overhead.

Listing 1. Complete PACS implementation.

```

1 import torch
2 import torch.nn as nn
3
4 class PACS(nn.Module):
5     """Precision-Aware_Confidence_Scaling."""
6
7     def __init__(self, tau=0.01, alpha=0.005,
8                 lambda_max=1.0, precision='fp16'):
9         super().__init__()
10        self.tau = tau
11        self.alpha = alpha
12        self.lambda_max = lambda_max
13        self.eps = {
14            'fp16': 2.0 ** -10,
15            'bf16': 2.0 ** -7,
16            'fp32': 2.0 ** -23,
17            'int8': 2.0 ** -7,
18        }[precision]
19
20    def forward(self, logits):
21        with torch.no_grad():
22            s = logits.abs().amax(dim=-1, keepdim=True)
23            r = torch.sigmoid((self.eps * s - self.tau) / self.alpha)
24            T = 1.0 + self.lambda_max * r
25        return logits / T

```

G.2. Integration Examples

Standard training loop. PACS integrates with a single line insertion after logit computation.

```

1 model = resnet50()
2 pacs = PACS(precision='fp16')
3
4 for images, labels in dataloader:
5     with torch.cuda.amp.autocast():
6         logits = model(images)
7         logits = pacs(logits) # Single line addition
8         loss = F.cross_entropy(logits, labels)
9     loss.backward()
10    optimizer.step()

```

Hugging Face Transformers. For language models, PACS integrates into the Trainer class(Wolf et al., 2020).

```

1 class PACSTrainer(Trainer):
2     def __init__(self, *args, **kwargs):
3         super().__init__(*args, **kwargs)
4         self.pacs = PACS(precision='bf16')
5
6     def compute_loss(self, model, inputs, return_outputs=False):
7         outputs = model(**inputs)
8         logits = self.pacs(outputs.logits)
9         loss = F.cross_entropy(
10             logits.view(-1, logits.size(-1)),
11             inputs['labels'].view(-1)
12         )
13         return (loss, outputs) if return_outputs else loss

```

H. Calibration Metrics

H.1. Expected Calibration Error

We use the standard binned ECE with 15 equal-width bins. Listing 2 provides our implementation.

Listing 2. ECE computation.

```

1 def compute_ece(probs, labels, n_bins=15):
2     confidences = probs.max(dim=1).values
3     predictions = probs.argmax(dim=1)
4     accuracies = (predictions == labels).float()
5
6     bin_boundaries = torch.linspace(0, 1, n_bins + 1)
7     ece = 0.0
8
9     for i in range(n_bins):
10         mask = (confidences > bin_boundaries[i]) & \
11             (confidences <= bin_boundaries[i + 1])
12         if mask.sum() > 0:
13             avg_conf = confidences[mask].mean()
14             avg_acc = accuracies[mask].mean()
15             ece += mask.float().mean() * (avg_conf - avg_acc).abs()
16
17     return ece.item() * 100 # Return as percentage

```

H.2. Additional Metrics

We also report Maximum Calibration Error (MCE), the maximum bin-wise calibration gap; Negative Log-Likelihood (NLL), which captures both accuracy and calibration; and Brier Score (Glenn et al., 1950), the mean squared error between predicted probabilities and one-hot labels.

I. Hardware Analysis

I.1. Computational Overhead

Table 14 presents microbenchmarks across GPU architectures. PACS adds 4–6 μ s per batch, representing less than 0.04% of the forward pass time.

Table 14. PACS computational overhead (batch size 256).

GPU	Forward (μ s)	PACS (μ s)	Overhead
A100	12,340	4.4	0.036%
V100	18,560	5.6	0.030%
RTX 3090	15,230	5.2	0.034%

I.2. Memory Analysis

PACS introduces no persistent memory overhead. Per-batch temporary allocations consist of three tensors of shape $(B, 1)$: scale s , risk r , and temperature T . For batch size 256 with FP32 storage, this totals $256 \times 1 \times 4 \times 3 = 3,072$ bytes (3 KB), negligible compared to model activations.

I.3. Operation Breakdown

The three PACS operations and their individual timings on A100:

Operation	Time (μ s)
<code>abs().amax()</code>	2.3
<code>sigmoid()</code>	0.9
<code>div()</code>	1.2
Total	4.4

J. Reproducibility

J.1. Environment

All experiments use PyTorch 2.1, CUDA 11.8, and torchvision 0.16. Vision models use torchvision pretrained weights; LLaMA uses official Meta checkpoints.

J.2. Random Seeds

We fix all random seeds for reproducibility:

```

1 import torch, numpy as np, random
2 torch.manual_seed(42)
3 torch.cuda.manual_seed_all(42)
4 np.random.seed(42)
5 random.seed(42)
6 torch.backends.cudnn.deterministic = True

```

J.3. Compute Resources

Table 15 summarizes computational requirements.

Table 15. Compute resources for all experiments.

Experiment	Hardware	GPU-hours
ImageNet training (3 seeds)	8× A100	288
CIFAR experiments	1× A100	24
Ablation studies	4× A100	192
LLaMA evaluation	2× A100	16
Total		~520

K. Statistical Significance

K.1. Confidence Intervals

Table 16 reports mean and standard deviation across three independent training runs with different random seeds.

K.2. Statistical Tests

We conduct paired t-tests comparing PACS against baselines using per-sample calibration errors. All comparisons show statistical significance at $p < 0.05$.

Table 16. ECE (%) with 95% confidence intervals across 3 seeds.

Method	ECE
Standard FP16	5.82 ± 0.21
LN	2.05 ± 0.12
PTS	2.08 ± 0.14
PACS	1.92 ± 0.08

Table 17. Paired t-test results (PACS vs baselines).

Comparison	<i>t</i> -statistic	<i>p</i> -value	Significant
PACS vs Standard	47.3	< 0.001	✓✓✓
PACS vs LN	2.84	0.023	✓
PACS vs PTS	3.12	0.018	✓